

Tunisian republic
Ministry of Higher Education
and Scientific Research
University of Sousse



Higher Institute of Applied
Sciences and Technology
of Sousse



IT DEPARTMENT

End of Studies Project Report

Presented to Obtain:

National Diploma of Computer Engineering
OPTION: SOFTWARE ENGINEERING-SOFTWARE ARCHITECTURE

**DevOps Chain construction to automate and
upgrade a traditional enterprise system**

Hosting Company



Elaborated by :

Mohamed Abdelaali Chachia

Presented on the: 11/07/2023 in front of the jury :

President : Mr. Houssemeddine Chtioui

ISSAT of SOUSSE

Examiner : Mrs. Nouha Khyari

ISSAT of SOUSSE

Academic supervisor : Mrs. Selma Belgacem

ISSAT of SOUSSE

Industrial supervisor : Mrs. Mouna Saidana

Bumbal

Academic Year : 2022/2023

Subject Code: FI-GL23-047

Abstract

THis work is part of the End of Studies Project, realized within the company BUMBAL, in order to obtain the national diploma of engineer in computer science. This project involves the implementation of the DevOps Chain to automate and enhance traditional enterprise systems, encompassing Continuous Integration, Continuous Deployment, Continuous Monitoring, Containerization, and container orchestration, the objective is to resolve existing issues and improve performance, ensuring high availability.

Keywords: DevOps ; Automation; Continuous Integration; Continuous Deployment; Continuous Monitoring; Containerization

Résumé

CE travail s'inscrit dans le cadre du Projet de Fin d'Études, réalisé au sein de l'entreprise BUMBAL, en vue d'obtenir le diplôme national d'ingénieur en informatique. Ce projet implique la mise en place de la Chaîne DevOps pour automatiser et améliorer les systèmes d'entreprise traditionnels, incluant l'Intégration Continue, le Déploiement Continue, la Surveillance Continue, la Conteneurisation, ainsi que l'orchestration de conteneurs, l'objectif est de résoudre les problèmes existants et d'améliorer les performances, garantissant une haute disponibilité.

Mots-clés : DevOps ; Automatisation ; Intégration Continue ; Déploiement Continue ; Surveillance Continue ; Conteneurisation.

Dedications

To my loving family who supported me through every step of this journey, and to my closest friends who believed in me and provided encouragement when I needed it the most.

To me, for always keeping my dreams in sight and never giving up, working hard for seven whole years without taking a break, I am proud.

This thesis is dedicated to all of you, without your love, support, and belief in me, this accomplishment would not have been possible.

Thank you from the bottom of my heart for being a part of my journey and for helping me turn my dreams into reality.

M. Abdelaali Chachia



Acknowledgement

At the end of this work, I am pleased to reserve these few lines of gratitude for all those who, from near or far, have contributed to the completion of this work.

I would like to express my heartfelt gratitude to Mrs. Selma Belgacem, my academic supervisor, for her invaluable guidance, support, and encouragement throughout my internship journey. Her unwavering commitment to my success and her wealth of knowledge has been instrumental in helping me complete this project.

I would also like to extend my sincere thanks to Mrs. Mouna Saidana, my industrial supervisor, for providing me with the necessary resources and support to complete this project. Her guidance and expertise in the field have been invaluable in helping me to achieve my goals. I cannot emphasize enough how genuinely grateful I am for her constant encouragement and mentorship.

To the whole team of Bumbal, thank you for your warm welcome and support throughout the entire internship period.

To all members of the honorable jury, whom I thank for agreeing to review this modest work.

Contents

List of Tables	xi
Abbreviations	xiii
General introduction	1
1 Preliminary study and project presentation	4
Introduction	5
1.1 Presentation of the company	5
1.2 Project presentation	7
1.2.1 Project Context	7
1.2.2 Problem and Motivation	8
1.2.3 Proposed Solution	9
1.2.4 Targets	10
1.3 Existing Solutions	11
1.4 Working Methodology	14
1.4.1 Development methodology : The Kanban Method	14
1.4.2 Modeling language: UML method	16
Conclusion	16
2 Requirements Specification	17

Introduction	18
2.1 Actors	18
2.2 Functional requirements	19
2.3 Non-functional requirements	20
2.4 Use case Diagrams	21
2.4.1 Global use case diagram	21
2.4.2 Continous integration/Continous deployment use case	24
2.4.3 Containers orchestration use case	27
2.4.4 Continuous monitoring use case	30
2.4.5 Logs management use case	32
Conclusion	34
3 Conception	35
Introduction	36
3.1 Software architecture	36
3.1.1 Global package diagram	36
3.1.2 Container Orchestration Package	38
3.1.3 Monitoring Stack Package	40
3.1.4 Logs Management System Package	41
3.1.5 Task Manager Package	42
3.1.6 DBCluster Package	43
3.2 Sequence diagrams	44
3.2.1 Create New Instance Module	44
3.2.2 Monitoring Module: Logging stack	46
3.2.3 Infrastructure monitoring stack	49
3.2.4 Automated Tasks Module	50

3.2.5	Load Balancers Module	52
	Conclusion	54
4	Realization	55
	Introduction	56
4.1	Development environment	56
4.1.1	Hardware environment	56
4.1.1.1	Development stage	56
4.1.1.2	Production stage	57
4.1.2	Software environment	58
4.1.2.1	PHPStorm	58
4.1.2.2	IntelliJ IDEA Ultimate	58
4.1.2.3	DataGrip	58
4.1.2.4	Bash	58
4.1.2.5	YAML	59
4.1.2.6	Groovy	59
4.2	Used technologies	59
4.2.1	Infrastructure As Code	59
4.2.1.1	Ansible	60
4.2.1.2	AWX Server	60
4.2.2	Continuous deployment and Containerization	60
4.2.2.1	Docker	60
4.2.2.2	Container orchestration with Kubernetes	61
4.2.2.3	Rancher Server	61
4.2.3	Continuous integration and Version control	61
4.2.3.1	Beanstalk	61

4.2.3.2	Jenkins	62
4.2.4	Continuous testing	62
4.2.4.1	Snyk	62
4.2.4.2	PHPStan	62
4.2.5	Database management	63
4.2.5.1	MariaDB	63
4.2.5.2	Galera Cluster	63
4.2.6	Continuous monitoring	64
4.2.6.1	ELK Stack	64
4.2.6.2	TIG Stack	65
4.2.7	Networking and Collaboration	66
4.2.7.1	Webhook	66
4.2.7.2	HAProxy	66
4.2.7.3	Discord	66
4.3	Physical architecture	67
4.4	Proof of concept	69
4.4.1	Kanban implementation	69
4.4.2	CI/CD pipeline	71
4.4.3	Containerization	78
4.4.3.1	Containers Orchestrated by Rancher	78
4.4.4	Continuous monitoring and automated problem fixing	79
4.4.4.1	Checking logs with Kibana	79
4.4.4.2	Monitoring environment resources usage	80
4.4.5	DevOps engineer administration	84
4.4.5.1	Create an app instance	85
4.4.5.2	Running instance interface	86

4.4.5.3	List of instances in backoffice	87
4.4.5.4	Docker images list	88
4.4.5.5	List of services in the backoffice	89
4.4.5.6	List of token used in backoffice	90
4.4.5.7	List of processes in the backoffice	91
4.4.5.8	AWX jobs running in the background	92
Conclusion		93

List of Figures

1.1	Bumbal Logo [1]	5
1.2	DevOps: The Continuous Loop [2]	7
1.3	Kanban Board [3]	15
2.1	Proposed solution global use case diagram	21
2.2	CI/CD use case diagram	24
2.3	Containers orchestration use case diagram	27
2.4	Continuous monitoring use case diagram	30
2.5	Logs management use case diagram	32
3.1	Global package diagram	36
3.2	Container Orchestration system component diagram	38
3.3	Monitoring system component diagram	40
3.4	Log management system component diagram	41
3.5	Task Manager component diagram	42
3.6	DBCluster component diagram	43
3.7	«Create New Instance» sequence diagram	44
3.8	"Logging stack" sequence diagram	46
3.9	"Infrastructure monitoring stack" sequence diagram	49
3.10	"Automation tasks Stack" sequence diagram	50

3.11 "Load Balancer" sequence diagram	52
3.12 How Load Balancer works	53
4.1 Global deployment diagram	68
4.2 Kanban Board (ClickUp tool)	69
4.3 Elk deployment task	70
4.4 Automate fix for galera cluster task	71
4.5 CI/CD Pipeline	72
4.6 PHPStan real-time error detection	73
4.7 Snyc real-time vulnerabilities detection	74
4.8 Development team notification	75
4.9 CI/CD pipeline real-time execution visualisation	76
4.10 Continuous Integration Trigger Configuration	77
4.11 Trigger Branch Configuration	77
4.12 Rancher's running containers interface	78
4.13 Kibana interface	79
4.14 Resources metrics interface in Grafana	80
4.15 Galera Cluster real-time metrics	81
4.16 Automated fixed resources problem	82
4.17 Self-fixed incident in DB cluster	83
4.18 Backoffice home page interface	84
4.19 Create an instance interface	85
4.20 Bumbal default instance interface	86
4.21 List of instances in the backoffice interface	87
4.22 Docker images list in the backoffice interface	88
4.23 Service list interface	89

4.24	List of credentials and tokens interface	90
4.25	Process list interface	91
4.26	remaining jobs running in backoffice	92
4.27	AWX dashboard interface	93
A1	Traditional Server Deployment [4]	100
A2	Docker Containers [4]	102

List of Tables

1.1	Comparison of DevOps platforms	13
2.1	Description of use case "Editing instance configuration"	23
2.2	Description of use case "Configure and manage pipelines"	26
2.3	Description of use case "Deploy and manage containerized applications"	29
2.4	Description of use case "Setup monitoring alerts"	31
2.5	Description of use case "Define log collection and ingestion pipeline" .	33
A1	Comparison between Traditional Server Deployment and Containers .	103

Abbreviations

TMS Transport Management System

SME Small Midsized Enterprises

SaaS Software as a Service

ETA Estimated Timing of Arrival

ERP Enterprise Resource Planning

API Application Programming Interface

K8S Kubernetes

UML Unified Modeling Language

SCM Source Code Management

General introduction

DevOps is a software engineering culture and practice that aims to incorporate software development (Dev) and software operation (Ops) by promoting collaboration, communication, and automation. The goal of DevOps is to deliver high-quality software faster, more reliably, and with greater efficiency. This is achieved by embracing a culture of continuous improvement, leveraging automation tools and processes, and fostering a close collaboration between developers and operations teams. DevOps practices emphasize the use of infrastructure as code, continuous integration, delivery, monitoring and feedback to increase the quality, and the reliability of the delivered software. DevOps helps organizations to meet the constantly evolving demands of the software industry and to keep pace with rapid technological changes.

Built upon these principles, our project takes a step further in leveraging technology advancements to enhance application deployment and management. Specifically, we focus on migrating an enterprise system from a shared pool VPS configuration to containerized environment, an advanced platform for managing containers. This migration opens up opportunities for organizations to achieve improved scalability, availability, efficient resource usage, simplified management, and streamlined deployment processes. With containers orchestration system, organizations can use the power of containerization and advanced orchestration of their software increments to optimize the software delivery pipelines and meet the challenges of the modern IT landscape.

Throughout this project, we will dive into the migration process, compare shared pool VPS configuration with containerized environment, and evaluate the impact on various aspects of application deployment and management. We will discover a range of tools and techniques suited to different applications and environments.

We will explore the concepts of Continuous Integration (CI), Continuous Testing, Continuous Deployment, and Continuous Monitoring to optimize our software development and delivery processes. In the CI phase, we will establish robust pipelines that facilitated seamless code integration, incorporating quality tests like unit tests, integration tests, and code analysis tools. This will promote frequent code integration and a culture of continuous improvement. For enhanced security, we will integrate a vulnerability scanning and testing capability into our CI/CD pipeline, scanning container images for vulnerabilities and taking immediate action to remediate any issues. In the Continuous Deployment stage, code changes will be automatically deployed to the appropriate environments after passing tests, streamlining the release cycle and enabling rapid feature delivery. To ensure optimal performance and proactive issue detection, we will implement a comprehensive monitoring system, tracking vital metrics and providing alerts and notifications for timely actions. This system will facilitate capacity planning and scalability, ensuring optimal user experience and service reliability.

In this manuscript, we present the steps we took to successfully complete this project. For that aim, we organized it into four chapters as follows:

The first one is aimed to present the hosting company and its application. We also present the general guidelines of our solution adapted to the company's demands.

The second chapter, "Requirement Specification", will primarily focus on outlining the requirements for the project. By modeling a use case diagram, those requirements will be defined and refined, and both functional and non-functional needs will be considered.

In the third chapter, "Architectural Design", we describe the proposed application's general and detailed architecture, and we design its functional behavior via sequence diagrams.

The fourth and final chapter of this manuscript outlines the primary technologies employed in the application development, along with the resulting architecture and user interfaces.

We finally conclude this manuscript with a synthesis of the reached goals at the end of this project. We also propose a set of enhancements that can be implemented as continuity of this work.

This project was done for the end of my studies for a national engineering degree from the Higher Institute of Applied Science and Technology of Sousse. It was conducted in collaboration with the Dutch company Bumbal.

The project's aim was to bridge the gap between theory and industry by applying innovative solutions and best practices in software engineering. It provided an opportunity to gain practical experience and learn from professionals at Bumbal.

Preliminary study and project presentation

Contents

Introduction	5
1.1 Presentation of the company	5
1.2 Project presentation	7
1.3 Existing Solutions	11
1.4 Working Methodology	14
Conclusion	16

Introduction

The main purpose of this first chapter is to place our project in its context. First, we present the company hosting the project. Then, we describe the analysis of the existing situation and the evaluation of similar solutions. We also list the objectives to be achieved at the end of this project. Finally, we explain the development method followed and the modeling language used.

1.1 Presentation of the company



Figure 1.1: Bumbal Logo [1]

Bumbal is a Netherlands-based company that has developed its own Transport Management System (TMS), known as Bumbal. The company focuses on implementing, developing, and publishing the software. Bumbal is used by Small Midsized Enterprises (SME), companies where the core business is not logistics. Bumbal's TMS works according to the Software as a Service (SaaS) principle and is completely offered online. In total there are now +200 companies actively using Bumbal's software locally and globally.

These companies are in the following target groups: construction, printing, electronics, bicycles, hospitality, couriers, garden/furniture, and health care. Customers can plan various things such as appointments, trips, deliveries. In addition, the customer

can use the track and trace and communicate with his end customers. Bumbal has been developed in such a way that it can be used not only with any Enterprise Resource Planning (ERP), TMS , or webshop system, but also with other systems thanks to the Application Programming Interface (API), which Bumbal uses.

The Bumbal application serves two purposes. Firstly, it calculates position to provide automatic Estimated Timing of Arrival (ETA) updates for both customers and planners. Secondly, it provides instructions to drivers and records the execution of these instructions.

Among other things, the following items are to be found in the application:

- Route schedules available for driver with sequence of stops, and within stops loading and service rules (what is to be delivered / collected or service visit).
- Loading lists per ride (survey of products that must be carried in that ride).
- Registration times of arrival and departure.
- Digital signature.
- Option to take pictures.
- Registration packaging.
- Scanning barcodes (also bulk scanning several barcodes at the same time).
- Processing driver instructions.
- Drivers registration remarks.
- Adding extra (non-planned) driver activities.

- Registration of irregularities (shortage of delivery / damage), with photo-signature inclusion if needed.

What is registered by the driver on the road, is also immediately available in the activities and ride dossier of the planner or other employees (sale or customer services). An e-mail can be automatically sent to staff (for instance customer service), mentioning the registered irregularities or shortages of delivery so that immediate action can be taken.

1.2 Project presentation

1.2.1 Project Context

DevOps is a transformative approach that brings together development and operations teams, fostering collaboration, efficiency, and agility within software development.



Figure 1.2: DevOps: The Continuous Loop [2]

Through the integration of DevOps practices such as continuous integration, con-

tinuous deployment, continuous testing, continuous monitoring, and automated infrastructure provisioning, DevOps empowers organizations to accelerate software deployment cycles while maintaining high-quality standards.

Migrating applications to a containerized environment has become increasingly popular in recent years due to its efficiency. Containers offer greater flexibility, scalability, portability, and high availability, allowing applications to be moved easily between different environments and hosting platforms, and ensuring minimal downtime in case of failures. This makes it easier to optimize resources and reduce costs, while also improving the agility and responsiveness of the application. However, migrating to a containerized environment can also be challenging, particularly for legacy applications(outdated computing software that is still in use) that were not designed for this type of environment.

1.2.2 Problem and Motivation

Previously, developers relied on operational teams to observe their code updates running on the staging environment, hindering their autonomy and causing delays. Moreover, code quality suffered from issues like lack of cleanliness and code smells, compromising maintainability and efficiency. Additionally, operational teams often required assistance from the DevOps Engineer to address anomalies while monitoring the infrastructure, leading to further bottlenecks in issue resolution.

The company also faced limitations and difficulties with the traditional method of deploying instance files on servers. This approach involved manual configuration and deployment of each instance, which proved to be time-consuming and error-prone, especially when dealing with a large number of instances. As the infrastructure scaled up, the management and scaling of instances became increasingly complex.

1.2.3 Proposed Solution

The proposed solution aims to address the challenges identified in the previous sections, providing effective remedies for the encountered problems.

To overcome the dependency on operational teams and the delays in code observation, the implementation of an automated DevOps chain is what we suggested.

DevOps chain implementation implies its practices implementation and automation as continuous testing, continuous integration, continuous deployment, continuous monitoring and infrastructure as code. Continuous deployment empowers developers to autonomously deploy and monitor their code updates in the staging environment, eliminating the need for external assistance and expediting the feedback loop.

To improve code quality, the proposed solution advocates the implementation of continuous integration and continuous testing practices. By automating the code integration process and running comprehensive tests throughout the development cycle, code cleanliness and the detection of code smells can be enhanced. This approach ensures that high-quality code is consistently produced, improving maintainability and efficiency.

Furthermore, to address the bottlenecks in issue resolution, the proposed solution suggests the adoption of comprehensive monitoring and alerting systems. Alongside this, the preparation of automated scripts to fix common anomalies can significantly enhance operational efficiency.

By leveraging real-time monitoring tools, automated alerts, and prepared scripts, operational teams can promptly identify and rectify any anomalies encountered during infrastructure monitoring. This enables faster and more efficient troubleshooting, reducing the reliance on the DevOps engineer and expediting issue resolution.

In addition to these measures, our proposed solution also includes the migration

of the application from traditional server deployment to a containerized environment managed by a container orchestration platform. This transition offers benefits such as efficient resource utilization, scalability, and improved management of the infrastructure. By leveraging containerization and a container orchestration platform, the company can streamline the deployment process, minimize resource wastage, and achieve significant cost savings.

Overall, the proposed solution emphasizes the importance of continuous deployment, continuous integration, continuous testing, and robust monitoring systems. By implementing these measures and preparing automated scripts, the company can foster developer autonomy, enhance code quality, streamline issue resolution processes, and significantly improve operational efficiency.

1.2.4 Targets

The proposed solution aims to accomplish a defined set of objectives that directly tackle the identified challenges and enhance the entire software development and deployment process. The primary objective is to empower developer autonomy and minimize reliance on operational teams. By adopting continuous deployment, developers acquire the capability to autonomously deploy and monitor their code updates in the staging environment, resulting in accelerated feedback loops and heightened agility.

Secondly, the aim is to improve code quality and maintainability. Through the implementation of continuous integration and continuous testing practices, the solution seeks to automate the code integration process and conduct thorough tests, ensuring the production of high-quality code with fewer bugs and code smells. This results in a more efficient and reliable codebase that is easier to maintain and enhance

over time.

Furthermore, the target is to streamline issue resolution and enhance operational efficiency. The adoption of comprehensive monitoring and alerting systems, along with the preparation of automated scripts, allows for proactive detection and swift resolution of anomalies during infrastructure monitoring. By reducing the reliance on manual interventions and expediting the resolution process, the proposed solution aims to minimize downtime and ensure the smooth functioning of the application.

Lastly, the ultimate goal is to achieve cost savings and resource optimization. The automation of infrastructure handling and administration enables efficient resource utilization and eliminates unnecessary expenses. By reducing resource wastage and improving scalability, the company can realize significant cost savings while maintaining high-quality software delivery.

In summary, the target of the proposed solution encompasses developer autonomy, improved code quality, streamlined issue resolution, and cost savings. By achieving these goals, the company can enhance its software development and deployment processes, leading to increased efficiency, better customer experiences, and a competitive edge in the market.

1.3 Existing Solutions

In this section, we will explore and compare several popular DevOps platforms offering different pipelines in order to realize the DevOps chain, including GitLab, Azure DevOps, and Bamboo.

We will examine their key features, capabilities, and differences, as shown in the table 1.1 to extract their drawbacks and incompatibilities with our client requirements and resolve them in our proposed solution.

- **GitLab :** [5] is a DevOps platform, providing seamless code management and version control. It enables developers to automate build, test, and deployment processes for their software projects.
- **Azure DevOps:** [6] is a cloud-based DevOps platform offered by Microsoft Azure. It provides a comprehensive set of tools and services for building, testing, and deploying applications across different environments.
- **Bamboo:** [7] is a robust DevOps platform that supports large-scale enterprise environments. It offers comprehensive deployment and release management capabilities, allowing teams to automate the build, test, and deployment processes in a secure manner.

Applications	Advantages	Disadvantages
Gitlab	• Easy setup and configuration. • Built-in containerization support.	• Not all DevOps practices are implemented. • Less flexibility in customization.
Azure DevOps	• Wide range of built-in services and integrations. • Scalable and highly available infrastructure.	• Not all DevOps practices are implemented. • Paid service with limited free tier capabilities. • Relatively complex configuration and setup process.
Bamboo	• Robust support for large-scale enterprise environments. • Strong security and access control features. • Comprehensive deployment and release management.	• Proprietary and not open source. • Steeper learning curve compared to other tools. • Requires additional licensing for advanced features.

Table 1.1: Comparison of DevOps platforms

1.4 Working Methodology

1.4.1 Development methodology : The Kanban Method

After a detailed study of different agile methodologies, we selected the Kanban method to manage our project, which is a lean method ensuring optimized use of our resources and best practices to enhance our product quality. In fact, this method aligns with our goal of maximizing efficiency and workflow. It guarantees continuous work progress and enables us to achieve optimal outcomes within a shorter time frame.

Kanban is a work method that is used to visualize and control the progress of a project, the tasks will be distributed in a Kanban board generally divided into three essential columns represent the different stages of a project, using cards to represent tasks: "To do", "Doing" and "Done" as shown in the figure 1.3. For each column of the board, a minimum and a maximum number of tasks is fixed. When the maximum number is reached, one task at least must be finished before another task can be added. This method emphasizes limiting work in progress and pulling work only when the team is ready to take it on, leading to better control of the project flow and more efficient work. It's often used in Agile software development

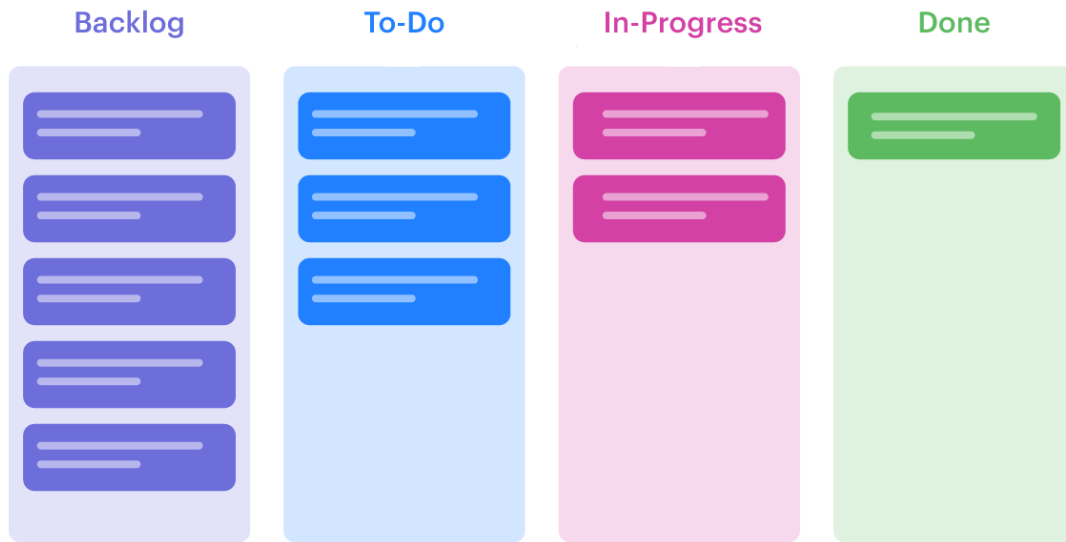


Figure 1.3: Kanban Board [3]

An overview of some of the planned primary tasks, which include:

1. Setting up and configuring a continuous integration pipeline.
2. Implementing a continuous deployment pipeline.
3. Implementing a continuous testing pipeline.
4. Setting up a database cluster.
5. Setting up a container orchestration cluster.
6. implementing an administration Infrastructure scripts
7. Migrating the deployment mechanism of delivered products to a containerized environment with automated orchestration for improved scalability and flexibility

8. Implementing automated scaling and load balancing
9. Implementing an automated fixing mechanism of detected delivered products anomalies

1.4.2 Modeling language: UML method

We adopted the Unified Modeling Language (UML) method to specify our client requirements and design our solution architecture and behavior. UML is a general purpose visual modeling language. It portrays the behavior and the structure of a software through two main categories of diagrams: static views as structural diagrams of use cases, class, object, component, deployment, and dynamic views as behavioral diagrams of activity, sequence, state-transition, and collaboration.

Conclusion

In this chapter we introduced the general context of the project by outlining its objectives with studying existing platforms, their contributions as well as their limits.

Requirements Specification

Contents

Introduction	18
2.1 Actors	18
2.2 Functional requirements	19
2.3 Non-functional requirements	20
2.4 Use case Diagrams	21
Conclusion	34

Introduction

This chapter focuses on analyzing the requirements of the customer for the proposed solution. The initial step involves identifying the actors who will interact with the application. Afterward, we define the functional and non-functional requirements followed by a detailed modeling of the software functionalities through use case diagrams.

2.1 Actors

An actor is an abstraction of a role performed by an external person or system that interacts directly with the software.

In our case, three main actors and a secondary actor are defined as follows :

- **Devops Engineer** : The principal actor responsible for managing the application's deployment in the containerized environment, as well as the development environment. He focuses on ensuring high availability and scalability of the application.
- **Developers** : They are responsible for coding the application and ensuring that it meets the functional and non-functional requirements of the stakeholders
- **Operationals** : They are responsible for monitoring and maintaining the infrastructure and ensuring the smooth operation of the deployed application.
- **Client assistant** : It is a secondary actor that helps the operationals to obtain the customers' preferences according to which they generate different application instances and manage them.

2.2 Functional requirements

To ensure a valuable and useful product, a clear and comprehensive list of functional requirements is necessary. The following are the identified functional needs:

- **Continuous integration:** Our solution should implement a continuous integration pipeline as a part of the development process, allowing developers to frequently merge and validate their code changes.
- **Continuous deployment:** Our solution should implement a continuous deployment pipeline that enables automated and efficient deployment of the application to the containerized environment, ensuring rapid and reliable releases.
- **Continuous testing:** Our solution should implement a continuous testing pipeline to automatically test the application at different stages of the development and deployment process, ensuring the quality and stability of the software.
- **Continuous monitoring:** Our solution should implement a continuous monitoring to proactively detect and address any performance or availability issues, providing real-time insights into the application's health and performance metrics.
- **Containerization:** Our solution should implement an automatic deployment mechanism using containers, enabling efficient resource utilization, isolation, and scalability.
- **Database Cluster:** Our solution should implement a robust database cluster to provide high availability, data redundancy, and scalability for the application's data storage and management.

2.3 Non-functional requirements

A non-functional requirement is a quality that software should have, like how fast it responds, how easy it is to use, and how safe it is. These things are important for the software to work well. To ensure the optimal performance and user experience of the application, we identified the following non-functional requirements:

- **Performance:** Our solution should ensure that applications running on the container orchestration system exhibit optimal performance, with minimal latency and response times.
- **Maintainability:** Our solution should ensure that containerized applications are easy to maintain and to manage, with straightforward troubleshooting and debugging processes.
- **Compatibility:** Our solution should ensure compatibility between the existing infrastructure, applications, and dependencies with the new containerization environment.

2.4 Use case Diagrams

2.4.1 Global use case diagram

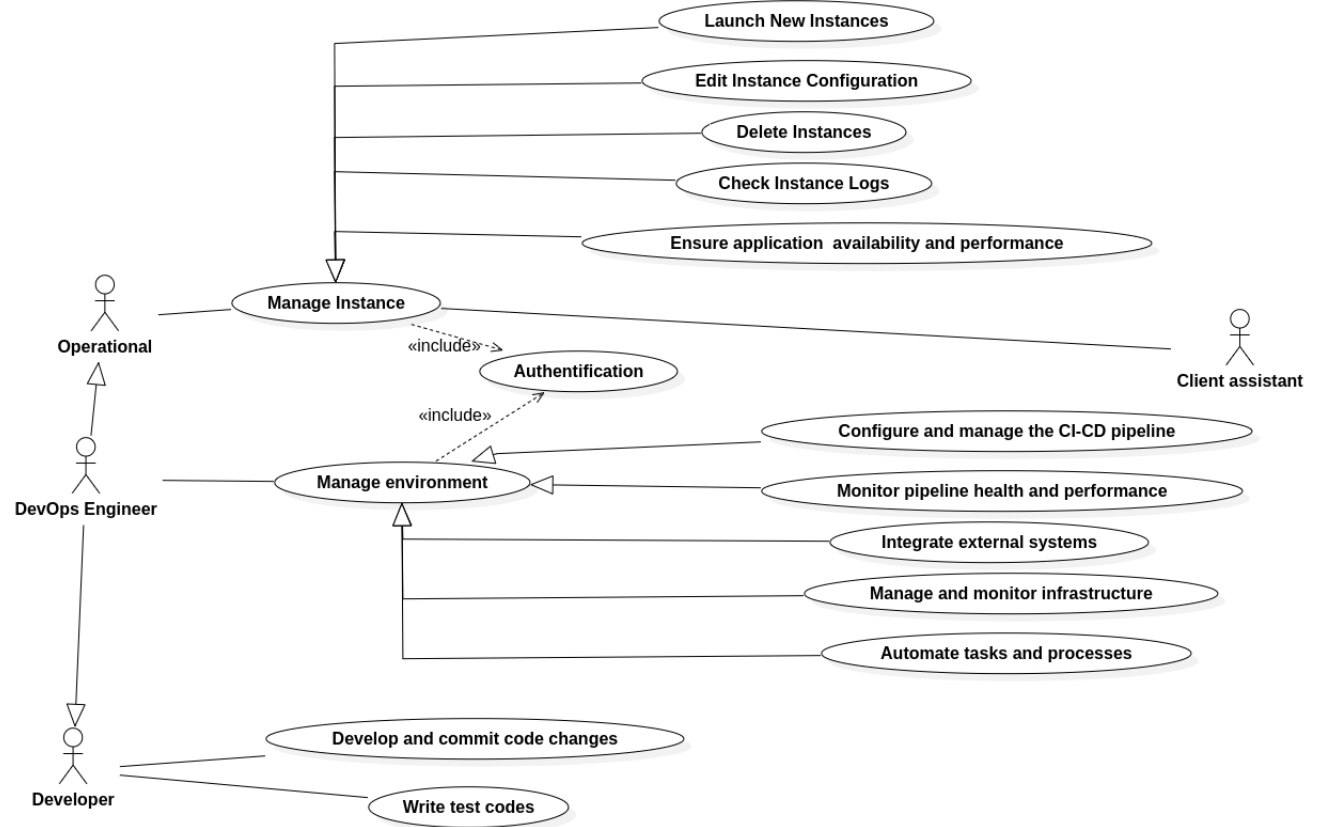


Figure 2.1: Proposed solution global use case diagram

- The operational staff can manage instances, which includes performing tasks such as launching new instances, editing existing instances, deleting instances, checking logs for instances, and ensuring application availability and performance. Authentication is required for them to perform these actions.
- The devops engineer can manage the environment, which involves configuring

and managing the CI/CD pipeline, monitoring its health and performance, integrating external systems, managing and monitoring infrastructure, and automating tasks and processes. Authentication is necessary for those tasks.

- The developer can commit code changes and write test code. They contribute to the development process by committing code changes to version control systems and writing test code to ensure software quality.

Editing instance configuration use case textual description: The table 2.1 shows the tasks to be performed by operational staff to edit instance configuration.

Summary	
Title	Editing instance configuration
Actors	Operational staff
Pre-condition	User is logged in
Post-condition	The configuration of the existing instance is successfully updated
Main scenario	• The operational selects the instance he wants to edit from the list of available instances. • The operational chooses the option to edit the configuration of the selected instance. • The system validates the operational's permissions and authentication. • If the authentication is successful and the editing action is allowed, the system presents the configuration settings for the selected instance. • The operational updates the desired configuration parameters, such as instance status, container Image version, API keys settings, or instance information. • The system verifies the validity of the new configuration and performs any necessary validations or checks. • If the configuration passes all validations, the system applies the updated configuration to the instance. • The system updates the configuration of the instance and reflects the changes in the application.
Exceptions	Conflicts with the new configuration, such as invalid input or incompatible settings

Table 2.1: Description of use case "Editing instance configuration"

2.4.2 Continuous integration/Continuous deployment use case

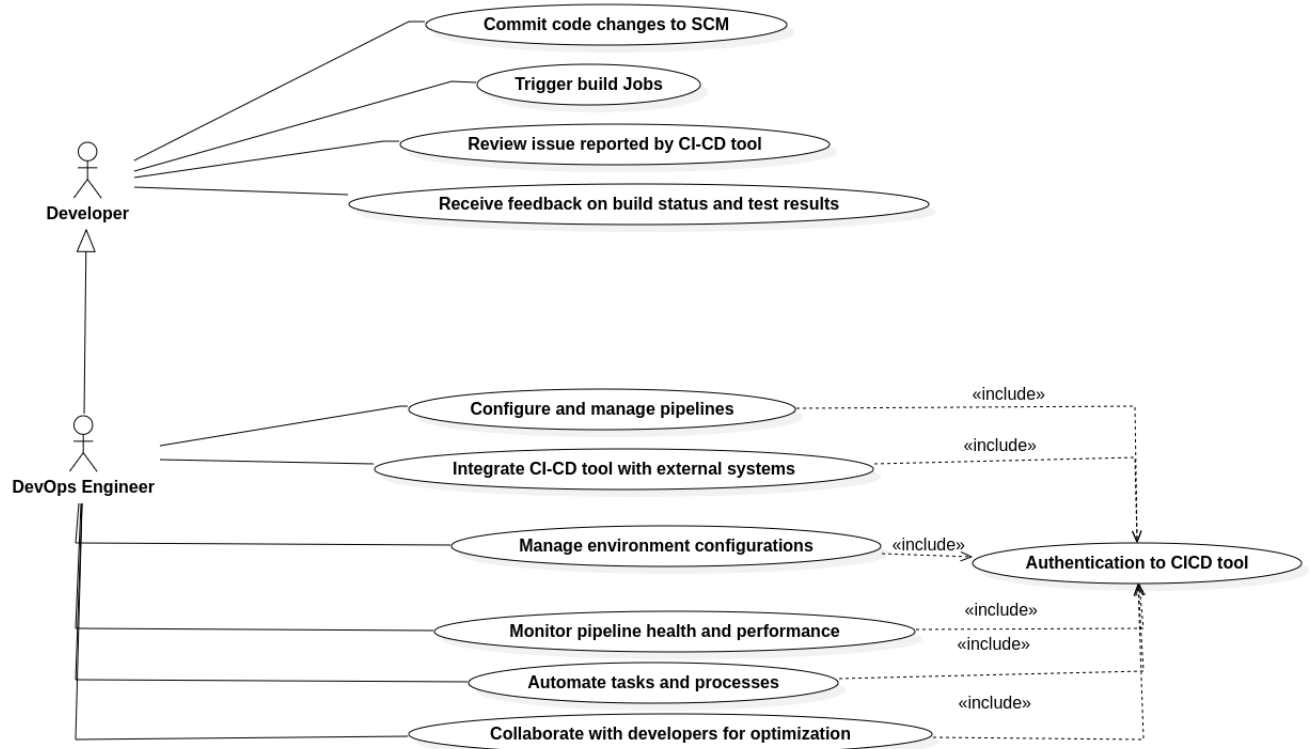


Figure 2.2: CI/CD use case diagram

- Developer is responsible for committing code changes to the Source Code Management (SCM) system. They can trigger build jobs to compile and package the code, review any issues reported by the CI/CD tool, and receive feedback on the build status and test results. This allows them to iterate and improve the code based on the CI/CD feedback.
- DevOps Engineer focuses on configuring and managing the CI/CD pipeline also integrating the CI/CD tool with external systems, managing environment configurations, monitoring the health and performance of the pipeline, automating

tasks and processes within the pipeline, and collaborating with developers to optimize the pipeline and ensure efficient software delivery. Authentication is necessary to access the CI/CD tools and perform these pipeline management tasks effectively.

Configure and manage pipelines use case textual description: The table 2.2 outlines the tasks required for the DevOps engineer in order to configure and manage pipeline.

Summary	
Title	Configure and manage pipelines
Actors	DevOps engineer
Pre-condition	User is logged in
Post-condition	The pipeline configuration is successfully updated and managed
Main scenario	• The DevOps engineer selects the pipeline he wants to configure and manage. • The system presents the configuration settings and options for the selected pipeline. • The DevOps engineer modifies the pipeline configuration parameters, such as stages, steps, triggers, and integrations. • The DevOps engineer can define pipeline triggers, such as code commits or scheduled intervals, to initiate pipeline execution. • The DevOps engineer manages the pipeline execution, including starting, pausing, or stopping the pipeline. • The system saves the updated configuration for the pipeline.
Exceptions	Conflicts with invalid configuration parameters, permission restrictions, or integration failures during the pipeline configuration and management process.

Table 2.2: Description of use case "Configure and manage pipelines"

2.4.3 Containers orchestration use case

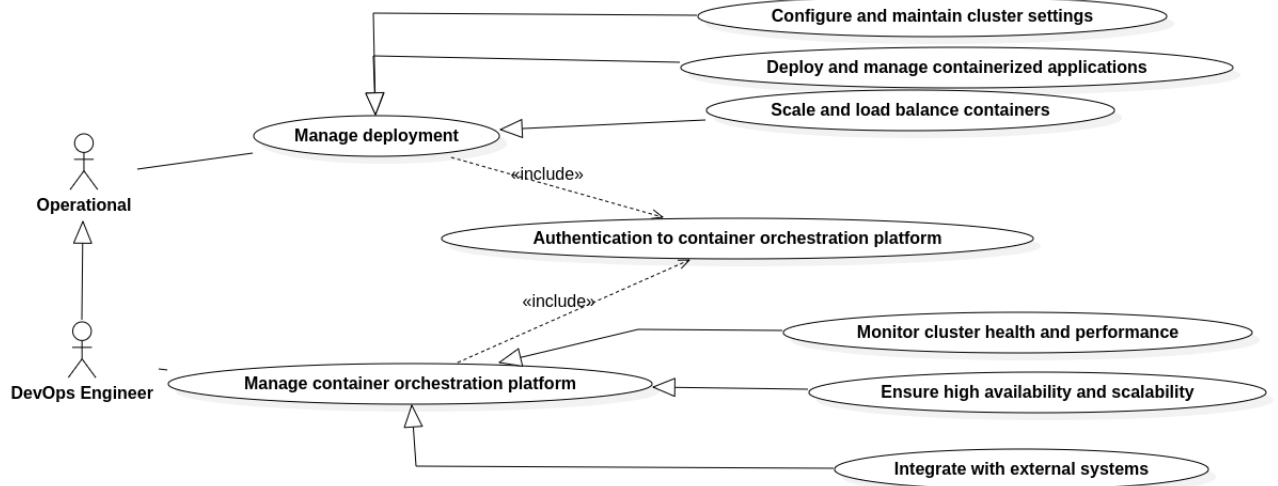


Figure 2.3: Containers orchestration use case diagram

- Operational staff can manage the deployment process, this includes configuring and maintaining cluster settings, deploying and managing containerized applications, as well as scaling and load balancing containers. Authentication is required to access the necessary tools and perform these deployment tasks.
- DevOps Engineer is responsible for managing the container orchestration platform, including tasks such as monitoring the health and performance of the cluster, ensuring high availability and scalability of the containers, also integrate the container orchestration platform with external systems. Authentication is necessary to access the container orchestration platform and perform those tasks.

Deploy and manage containerized applications use case textual description: The table 2.3 provides an overview of the tasks required for deploying

and managing containerized applications by the operational staff.

Summary	
Title	Deploy and manage containerized applications
Actors	Operational
Pre-condition	User is logged in
Post-condition	The containerized applications are successfully deployed and managed in the desired environment
Main scenario	• The operational selects the target environment for deploying the containerized application. • The operational identifies the required container images or builds new ones according to the application requirements. • The operational configures the necessary settings for the deployment, such as resource allocation, networking, and environment variables. • The operational initiates the deployment process by using the container orchestration platform. • The system verifies the availability of the required resources and dependencies. • If all prerequisites are met, the containerized application is deployed to the specified environment. • The operational monitors the deployment process to ensure its successful completion. • the operational manages the containerized application, such as updating or performing rolling restarts.
Exceptions	The deployment process may be affected by insufficient resources, configuration errors or conflicts, as well as network or connectivity issues.

Table 2.3: Description of use case "Deploy and manage containerized applications"

2.4.4 Continuous monitoring use case

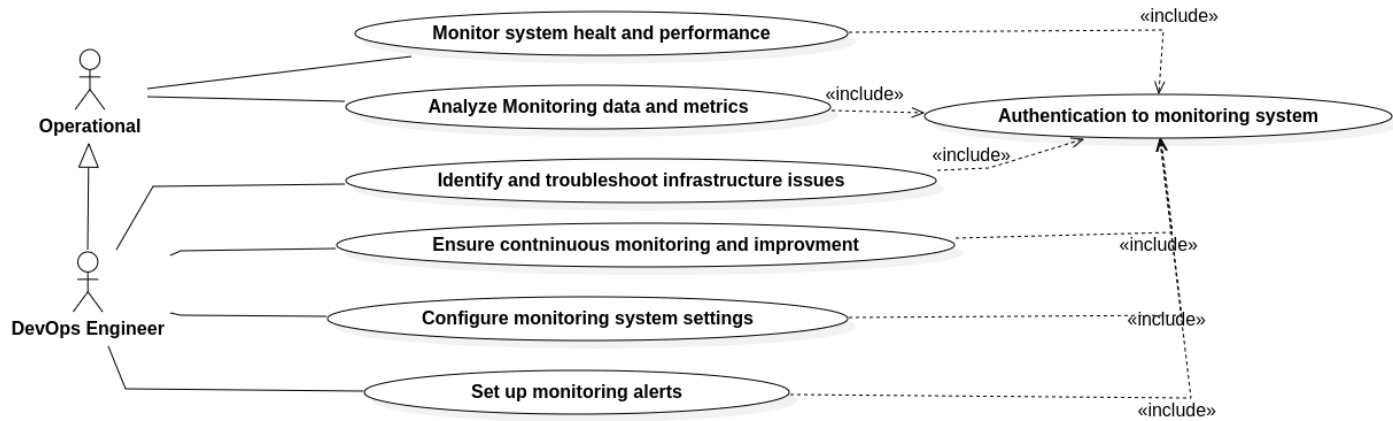


Figure 2.4: Continuous monitoring use case diagram

- Operational staff is authorized to monitor the health and performance of the system. They can analyze monitoring data and metrics to gain insights into the system's behavior. Authentication is required to access the monitoring tools and view system health and performance information.
- DevOps engineer is responsible for identifying and troubleshooting infrastructure issues. He ensures continuous monitoring of the infrastructure, configures monitoring system settings, and sets up monitoring alerts for timely notifications. Authentication is necessary to access the monitoring system and perform these tasks effectively.

Setup monitoring alerts use case textual description: The table 2.4 outlines the tasks involved in setting up monitoring alerts by the DevOps engineer.

Summary	
Title	Setup Monitoring Alerts
Actors	DevOps Engineer
Pre-condition	User is logged in
Post-condition	The system sends notifications or alerts to the designated recipients when the configured thresholds are breached.
Main scenario	• The DevOps engineer identifies the components or metrics that require monitoring. • The DevOps engineer configures monitoring rules and thresholds for the identified components or metrics. • The monitoring system verifies the validity of the configured rules and thresholds. • If the configuration passes all validations, the monitoring alerts are activated.
Exceptions	Incorrect or invalid configuration settings for monitoring rules and thresholds, failure to activate monitoring alerts, or delivery failures to designated recipients.

Table 2.4: Description of use case "Setup monitoring alerts"

2.4.5 Logs management use case

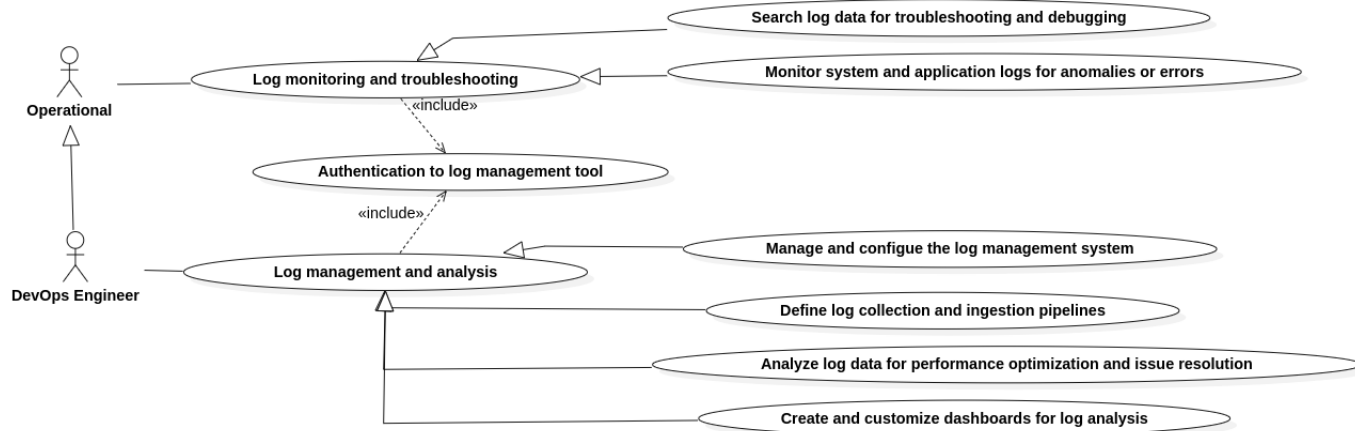


Figure 2.5: Logs management use case diagram

- Operational staff, with proper authentication, is authorized to monitor logs and troubleshoot issues. They can search log data for troubleshooting and debugging purposes, as well as monitor system and application logs to identify any anomalies or errors.
- Devops engineer is authorized to manage logs and perform log analysis. He has the responsibility to manage and configure the log management system, define the log collection and ingestion pipeline, analyze log data for performance optimization, and create customized dashboards for log analysis. Authentication is necessary for him to access the log management and analysis tools.

Define log collection and ingestion pipeline: The table 2.5 provides an overview of the tasks required for defining the log collection and ingestion pipeline

by the DevOps engineer.

Summary	
Title	Define log collection and ingestion pipeline
Actors	DevOps Engineer
Pre-condition	User has access to the application's codebase and the necessary permissions to configure log collection.
Post-condition	The log collection process is successfully defined and integrated with the application code.
Main scenario	• The DevOps engineer identifies the log data sources within the application code. • The DevOps engineer modifies the application code to include logging statements and libraries to capture the required log data. • The DevOps engineer configures the log format and log levels according to the logging requirements. • The system launches the the log collection process automatically upon running the application. • The system sends the log data to the processing component for further analysis.
Exceptions	Incorrect modification of the application code, resulting in ineffective log collection, or misconfiguration of the log format or log levels, leading to inconsistent or insufficient log data.

Table 2.5: Description of use case "Define log collection and ingestion pipeline"

Conclusion

In this chapter, we inspected the project's functional and non-functional requirements and presented the global use case diagram and a detailed description of primary use cases.

Contents

Introduction	36
3.1 Software architecture	36
3.2 Sequence diagrams	44
Conclusion	54

Introduction

This chapter outlines a suitable structure for our solution. It serves as a fundamental stage in constructing the project, aiming to establish a solid groundwork for implementation. To achieve this, we define the overall design of our solution, followed by a detailed examination of its components and behavior.

3.1 Software architecture

3.1.1 Global package diagram

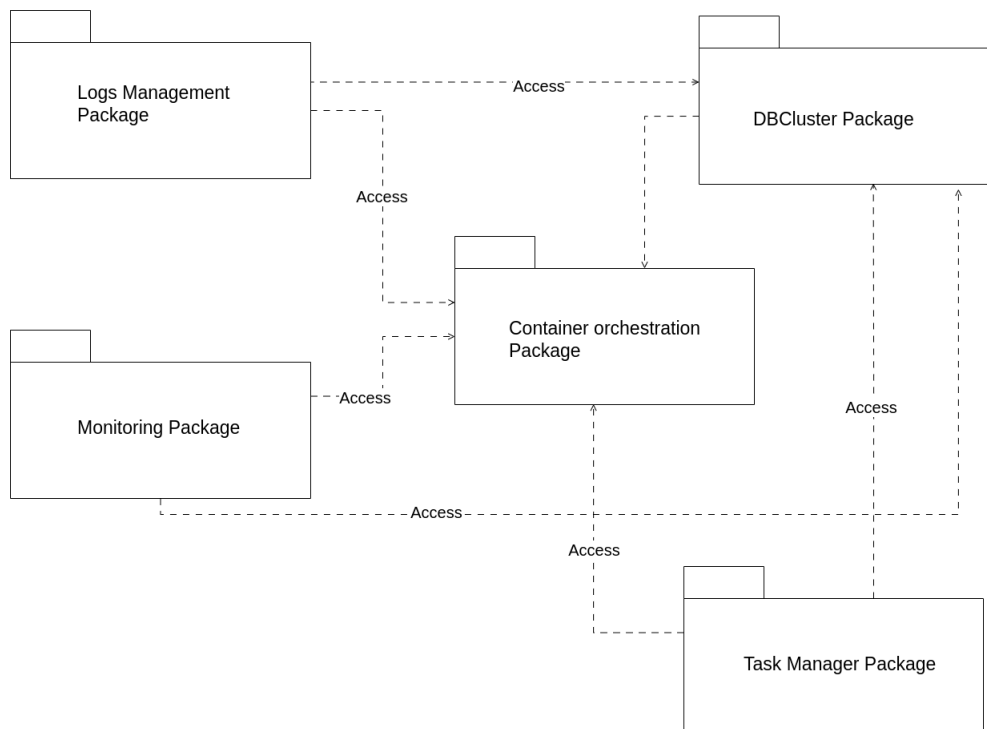


Figure 3.1: Global package diagram

The **Logs Management Package** is responsible for collecting, processing, storing, and analyzing log data. It encompasses components such as the log collector, processing component, log storage or indexing system, and data visualization tool.

The **DBCluster Package** is responsible for managing and maintaining a database cluster. It includes components for database replication, data synchronization, failover mechanisms, and load balancing.

The **Monitoring Package** takes charge of monitoring the performance and health of the application and infrastructure. It comprises components for collecting metrics, setting up alerts and notifications, and generating reports on system behavior.

The **Container Orchestration Package** is responsible for managing and orchestrating containerized applications. It encompasses components for deploying, scaling, and managing containers, as well as load balancers and auto-scaling mechanisms.

Finally, the **Task Manager Package** handles the execution and management of automated tasks and workflows. It includes components such as automation scripts, inventory management, and task execution on hosts.

3.1.2 Container Orchestration Package

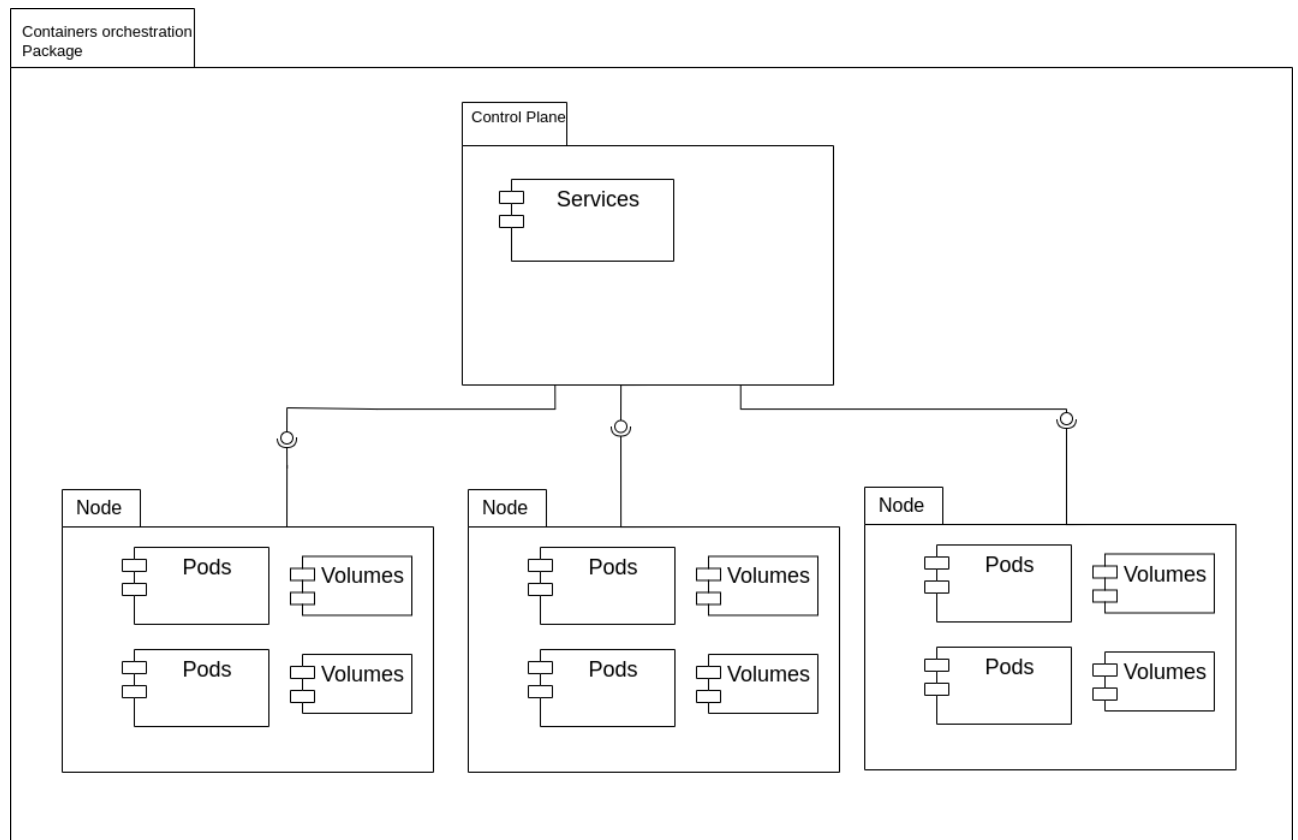


Figure 3.2: Container Orchestration system component diagram

Container orchestration, such as Kubernetes (K8S) , employs a distributed architecture to manage containerized applications across a cluster of nodes. The key components of a container orchestration system include:

- **Control Plane:** The control plane consists of several components that manage and coordinate the cluster. It includes the API server, which serves as the central management point for interacting with the cluster. Other components

like the scheduler, controller manager, and etcd provide functionalities like scheduling containers, managing cluster state, and storing configuration data.

- **Nodes:** Nodes are the worker machines in the cluster responsible for running the containers. Each node has the necessary tools and services to manage containers, including a container runtime, such as Docker. Nodes communicate with the control plane and receive instructions on scheduling, deploying, and scaling containers.
- **Pods:** Pods serve as the smallest deployable entities within a container orchestration platform. They encapsulate one or more containers and provide shared resources like networking and storage. This allows closely located containers to effectively communicate and share resources, promoting efficient collaboration within the platform.
- **Services:** Services provide network connectivity and load balancing for the containers within the cluster. They enable containers to be accessed internally or exposed to external traffic using different types of service definitions.
- **Volumes:** Volumes provide persistent storage for containers. They allow data to be stored and shared across containers and survive container restarts. Different types of volumes, such as hostPath, emptyDir, or network-based storage, can be used based on the specific requirements.

3.1.3 Monitoring Stack Package

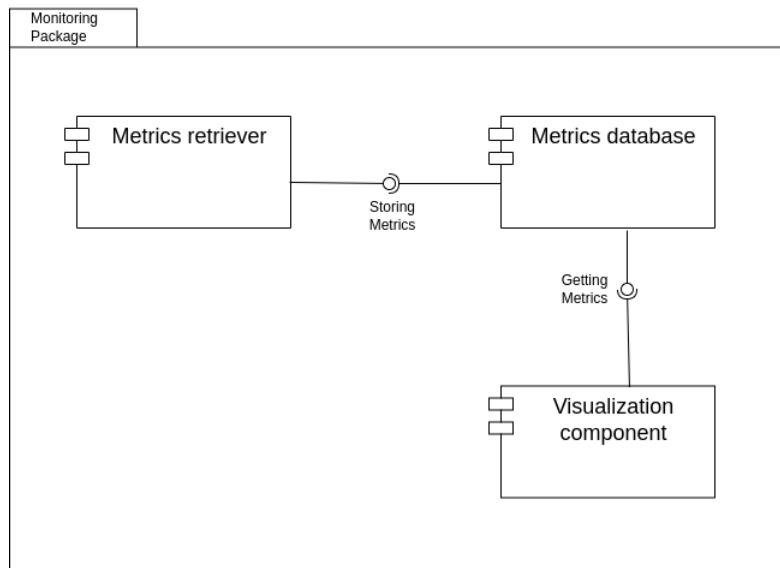


Figure 3.3: Monitoring system component diagram

The monitoring system architecture typically consists of several components working together to collect, store, and visualize data. One essential component is the **metrics retriever**, responsible for collecting metrics from various sources such as systems, devices, and applications. These metrics are then processed and stored in a dedicated **database** or storage system. The **visualization** or dashboarding component provides a user-friendly interface to visualize the collected metrics, enabling users to monitor and analyze the data in real-time.

3.1.4 Logs Management System Package

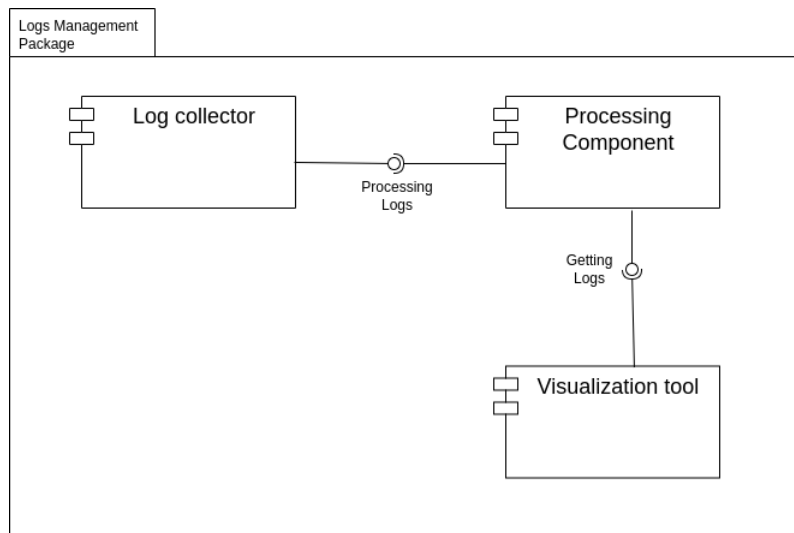


Figure 3.4: Log management system component diagram

The architecture of a logs management system involves multiple components working together to collect, process, store, and analyze log data. The **log collector** gathers logs from different sources and forwards them to a processing component. This **processing component** applies filters and transformations to the logs and stores them in a log storage or indexing system. The logs can then be queried and analyzed using a **visualization** or analytics tool, allowing users to gain insights into system behavior and troubleshoot issues.

3.1.5 Task Manager Package

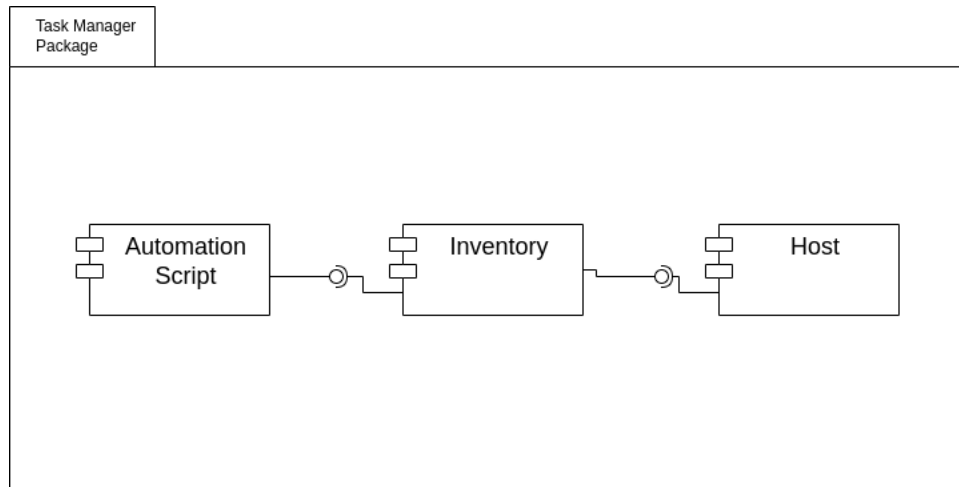


Figure 3.5: Task Manager component diagram

The Task Manager system operates through the collaboration of three main components. The **Automation script**, serving as the core functionality, is responsible for executing tasks. It retrieves **host** information from the **Inventory** component, which acts as a repository of host data. Using this information, the Automation script establishes connections with the respective hosts and carries out the assigned tasks. Once completed, the hosts send back the results to the script for further processing. The Task Manager also collects and stores host facts, which are saved in the inventory for future reference. This enables the Inventory component to provide the Task Manager with the necessary host facts whenever required, ensuring efficient and informed task management.

3.1.6 DBCluster Package

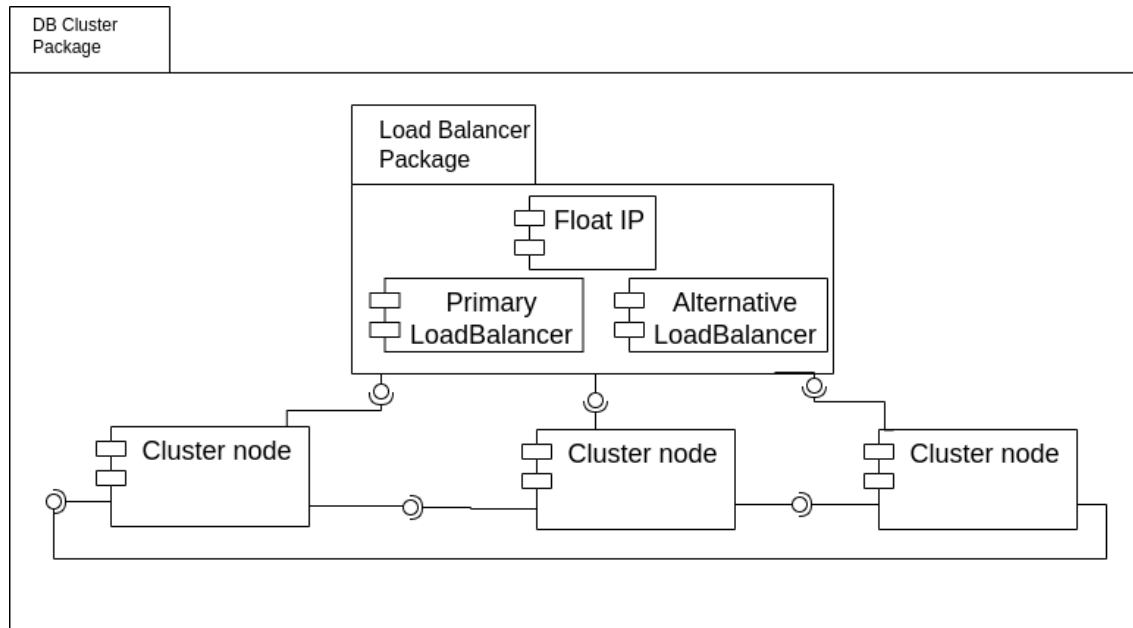


Figure 3.6: DBCluster component diagram

The architecture of a database cluster involves two main components: the **Cluster Nodes** and the **Load Balancer**. The Cluster Nodes are responsible for storing and processing data within the cluster. Each node is a database server that handles data storage, retrieval, and execution of queries. The Load Balancer component plays a crucial role in distributing incoming database requests across the cluster nodes. It ensures that the workload is evenly distributed, optimizing resource utilization and enhancing performance. By intelligently routing requests to the appropriate nodes, the Load Balancer helps preventing overloading and promotes scalability and fault tolerance within the database cluster. Together, these components form a resilient and efficient database cluster architecture, enabling high availability and reliable data access for applications.

3.2 Sequence diagrams

3.2.1 Create New Instance Module



Figure 3.7: «Create New Instance» sequence diagram

- **Check Domain Availability:** The app checks if the instance already exists through Web hosting service.
- **If the instance is already available:**
 - The app sends an error message indicating that it already exists.
- **Else:**
 - **Add Database (DBCluster Manager):** The app uses the DBCluster Manager to add a new database to the Database cluster.
 - **Add Domain to LoadBalancer (Task Manager):** The app uses the Task Manager to add the domain to the load balancer.
 - **Check if AddDomain is finished:** The app checks if the previous task of adding the domain to the load balancer is finished.
 - **Add Namespace to container orchestration platform:** The app adds a new namespace to the container orchestration platform.
 - **Add Containers to container orchestration platform:** The app use a container orchestration platform to add containers.
 - **Check if AddContainers is finished:** The app checks if the previous task of adding containers is finished.
 - **Check if the container is running and reachable:** The app verifies if the container is running and reachable.
 - **Add Routes to Load Balancer:** The app adds new routes to the load balancer.
 - **Add new Configuration to the Instance:** The app adds a new configuration to the instance.

- **Add new user to the Instance:** The app adds a new user to the instance.
- **Final Check on 200 API Response:** The app performs a final check on the 200 API response to ensure successful completion.

3.2.2 Monitoring Module: Logging stack

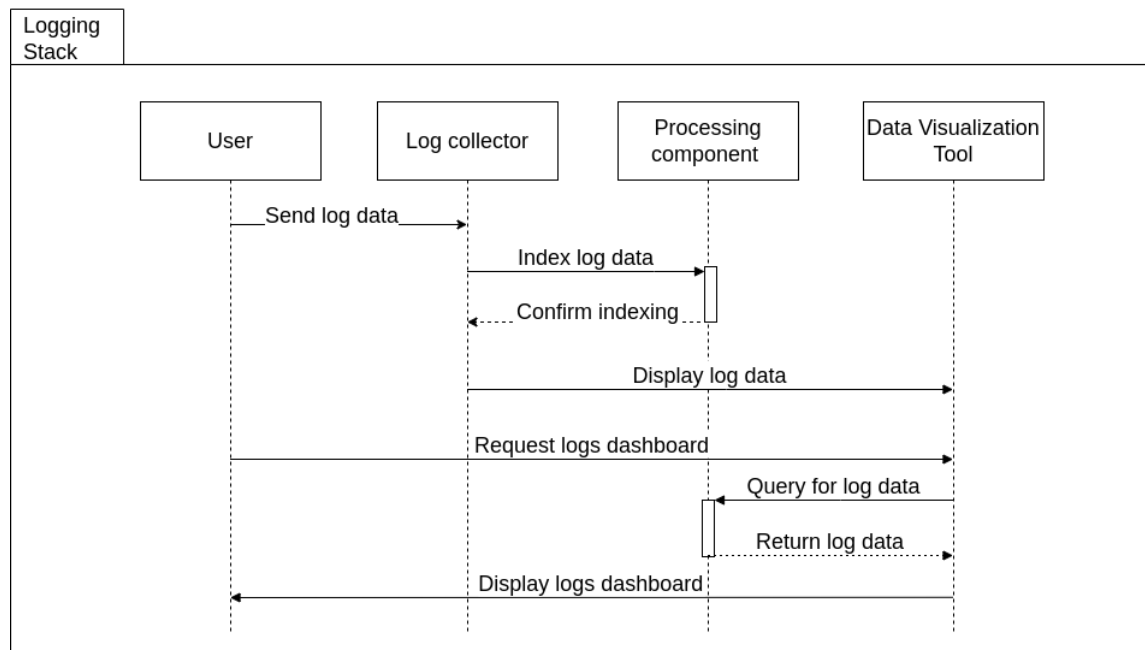


Figure 3.8: "Logging stack" sequence diagram

- **User sends logs to Log collector:** The logging stack allows users to seamlessly integrate their applications with the Log collector, and the logs generated during runtime are automatically sent to the Log collector for processing and analysis.

- **Log collector indexes log data to Processing component:** Once the Log collector receives the log data, it indexes it into the Processing component, a distributed search and analytics engine. This indexing process organizes the log data in a structured manner, making it easily searchable and accessible for subsequent analysis.
- **Log collector indexes log data to Processing component:** Once the Log collector receives the log data, it indexes it into the Processing component, a distributed processing and analytics engine. This indexing process organizes the log data in a structured manner, making it easily searchable and accessible for subsequent analysis.
- **Processing component confirms indexing:** Upon successful indexing, the Processing component confirms the completion of the process. This confirmation ensures that the log data is securely stored and readily available for querying and analysis, providing assurance that the data is captured and ready for use.
- **Data Visualization Tool displays log data:** The Data Visualization Tool, a component of the logging stack, provides a user-friendly interface for interacting with log data. It retrieves the indexed log data from the Processing component and presents it in a visually appealing and customizable format, allowing users to explore and analyze their log data easily.
- **User requests logs dashboard from Data Visualization Tool:** When a user requests a logs dashboard from the Data Visualization Tool, he initiates a specific query or search for the desired log data. This request prompts the Data Visualization Tool to query the Processing component for the relevant

log data needed to populate the requested dashboard.

- **Data Visualization Tool queries for log data from Processing component:** The Data Visualization Tool sends a query to the Processing component, specifying the parameters, filters, and criteria to retrieve the log data required for the logs dashboard. The Processing component processes the query and searches its indexed log data based on the provided search parameters.
- **Processing component returns log data to the Data Visualization Tool:** Upon receiving the query, the Processing component searches its indexed log data and retrieves the relevant log entries that match the specified criteria. It then returns the retrieved log data to the Data Visualization Tool for further processing and visualization.
- **Data Visualization Tool displays logs dashboard to the user:** Finally, the Data Visualization Tool receives the log data from the Processing component and generates a visually interactive logs dashboard. This dashboard includes charts, tables, and graphs, presenting the log data in a comprehensive and user-friendly manner. Users can explore, analyze, and gain valuable insights from the displayed logs, facilitating troubleshooting, monitoring, and decision-making processes.

3.2.3 Infrastructure monitoring stack

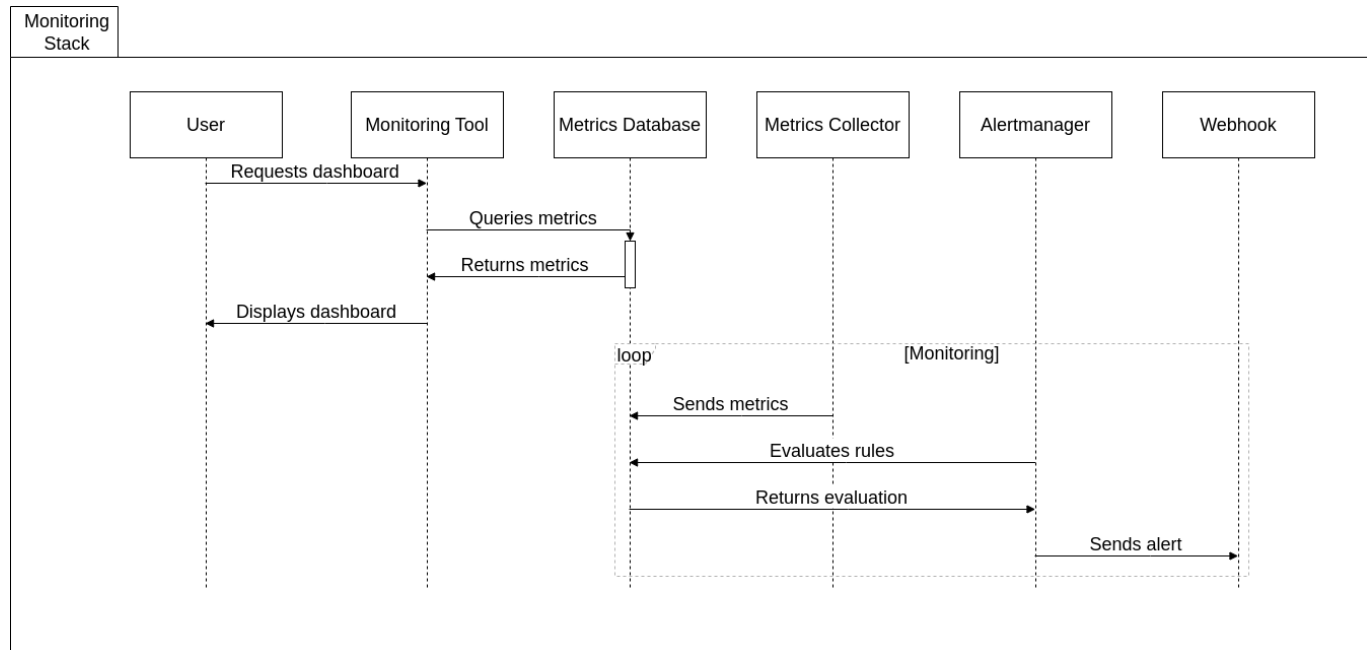


Figure 3.9: "Infrastructure monitoring stack" sequence diagram

- **Monitoring Tool queries metrics from Metrics database:** The Monitoring Tool retrieves metrics from the Metrics database, a high-performance storage system for time series data, based on the user's request.
- **Metrics database returns metrics to the Monitoring Tool:** The Metrics database processes the query and returns the relevant metrics data to the Monitoring Tool for visualization and analysis.
- **Monitoring Tool displays the dashboard to the user:** Using the retrieved metrics data, the Monitoring Tool generates an interactive dashboard with visualizations and charts for the user to explore.

- **Metrics Collector sends metrics to Metrics database:** The Metrics Collector collects metrics data from various sources and periodically sends it to the Metrics database, ensuring a continuous flow of real-time data.
- **Alertmanager evaluates rules in Metrics database:** The Alertmanager evaluates predefined rules and conditions based on the metrics data stored in the Metrics database.
- **Metrics database returns the evaluation to the Alertmanager:** The Metrics database provides the evaluation results of the metrics data to the Alertmanager for further analysis.
- **Alertmanager sends alert to webhook:** If a rule or condition is met, the Alertmanager sends an alert to a configured webhook for notification purposes.

3.2.4 Automated Tasks Module

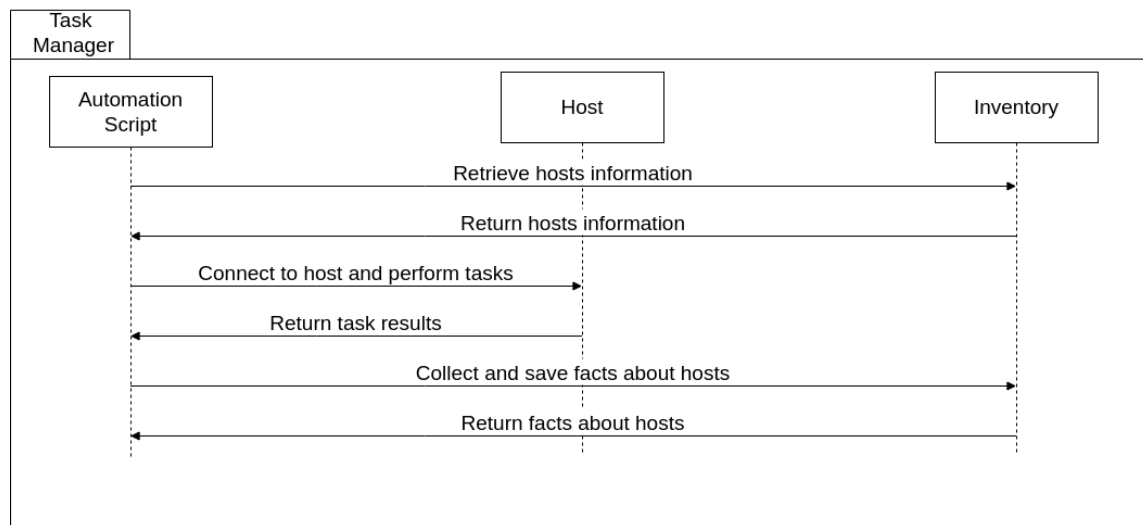


Figure 3.10: "Automation tasks Stack" sequence diagram

- **Task Manager runs Automation script:** The Task Manager executes the Automation script, which retrieves host information from the inventory.
- **Inventory returns host information:** The inventory provides host information to the Automation script for further processing.
- **Automation script connects to hosts and performs tasks:** The script connects to hosts and executes defined tasks.
- **Host returns task results:** After executing tasks, the host sends back the results to the script.
- **Task Manager collects and saves host facts:** The Task Manager gathers and stores facts about hosts in the inventory.
- **Inventory returns host facts:** The inventory provides the stored host facts to the Task Manager for future use.

3.2.5 Load Balancers Module

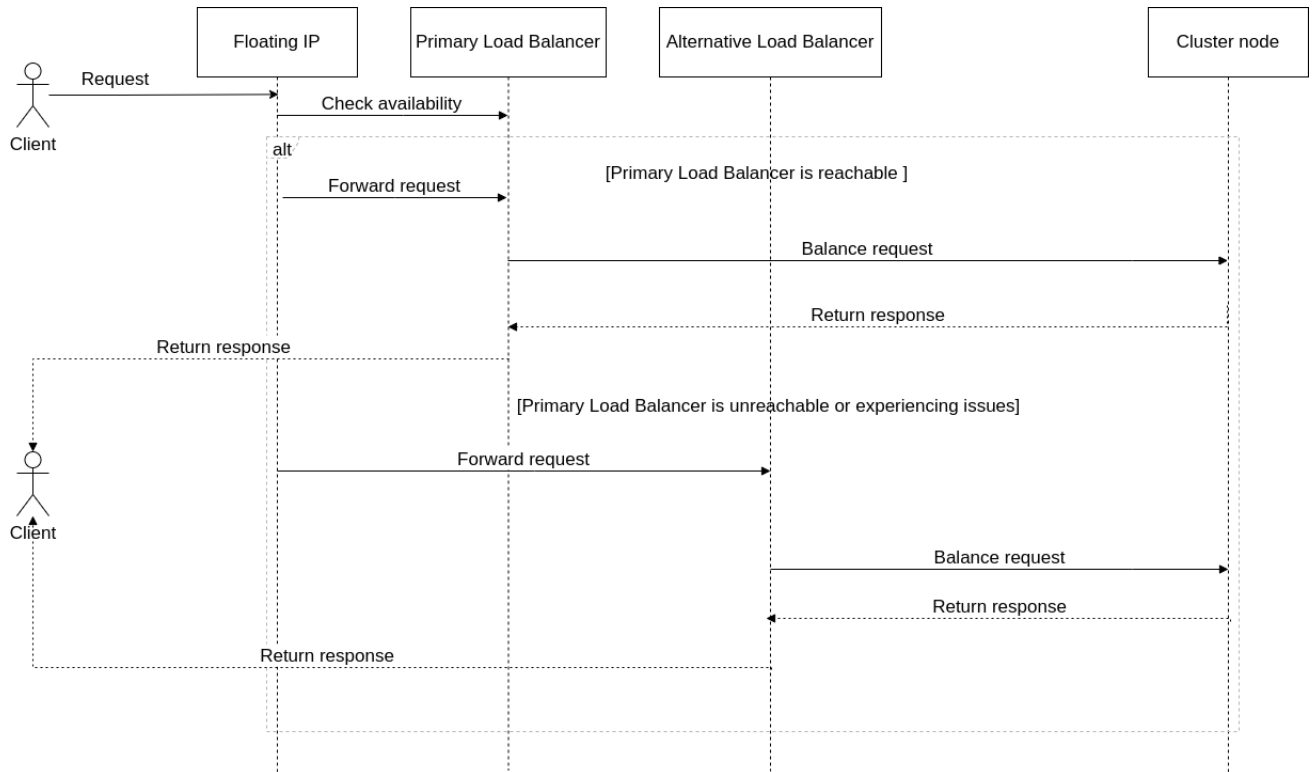


Figure 3.11: "Load Balancer" sequence diagram

- **Floating IP checks primary load balancer:** The floating IP verifies if the primary load balancer is unreachable or experiencing issues.
- **Floating IP forwards request to alternative healthy load balancer:** If the primary load balancer is unreachable, the floating IP redirects the request to an alternative load balancer that is in a healthy and operational state.
- **Load balancer receives the request:** The selected load balancer, either the primary or an alternative one, receives the request.

- **Load balancer load balances the request:** The load balancer distributes the request to one of the cluster nodes based on the configured load balancing algorithm.
- **Cluster node processes the request:** The selected cluster node executes the requested tasks.
- **Cluster node sends the response:** The cluster node generates a response containing the requested data or the result of the operation.
- **Load balancer receives the response:** The load balancer receives the response from the cluster node.
- **Load balancer forwards the response to the client:** The load balancer sends the response back to the original client that made the initial request.

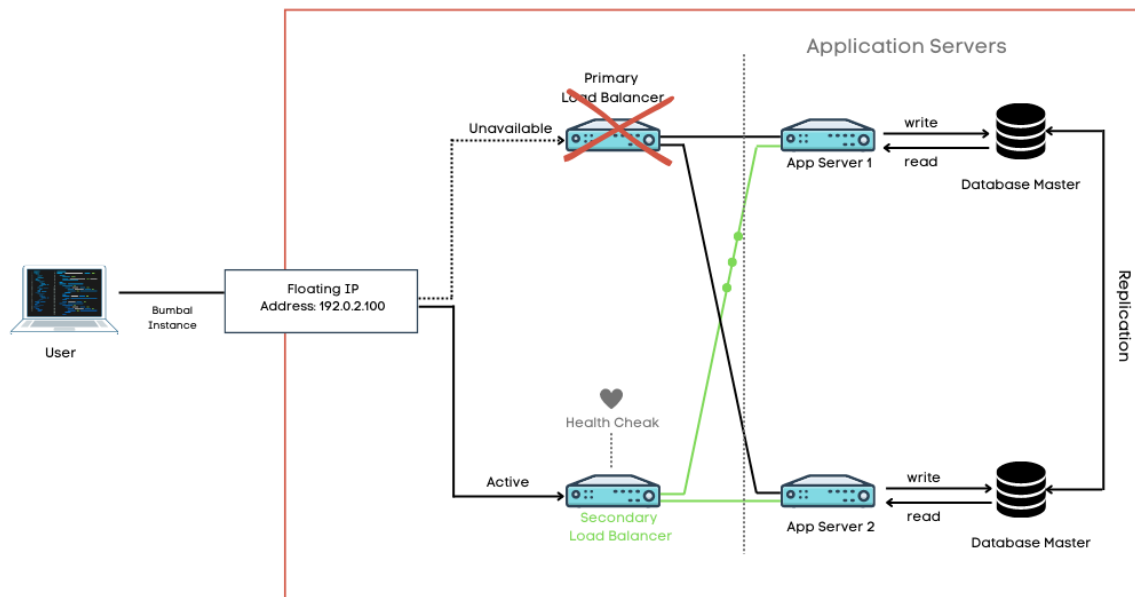


Figure 3.12: How Load Balancer works

Conclusion

In this chapter we provided an overview of the DevOps platform architecture and we highlighted the key components and their roles. In the following chapter we will present the implementation phase and exhibit the work we realised.

Contents

Introduction	56
4.1 Development environment	56
4.2 Used technologies	59
4.3 Physical architecture	67
4.4 Proof of concept	69
Conclusion	93

Introduction

The conception phase is primordial for a clear and more rigorous vision of an optimized architecture’s implementation. We devote this chapter to illustrate the discussed functionalities of our DevOps chain, and to detail the used implementation tools and environments.

4.1 Development environment

The hardware environment and the software environment used to attend the completion of this project are described in this section.

4.1.1 Hardware environment

4.1.1.1 Development stage

During the various stages of our project, namely design, documentation, code implementation, and testing, we arranged a personal computer configured as follows :

Specification	Value
Brand	HP OMEN
Processor	Intel CORE i7 7th Gen
RAM	12 GB
Operating System	Linux, Ubuntu 20.04 LTS

4.1.1.2 Production stage

Host	Operating System	Specifications
Jenkins Build Node	CentOS 7.5 (64 bit)	Memory: 12 GB, Disk Space: 128 GB
Beanstalk SCM	CentOS 7.1 (64 bit)	Memory: 12 GB, Disk Space: 256 GB
Grafana	CentOS 7.1 (64 bit)	Memory: 8 GB, Disk Space: 200 GB
InfluxDB	CentOS 7.1 (64 bit)	Memory: 60 GB, Disk Space: 500 GB
Rancher UI	CentOS 7.1 (64 bit)	Memory: 8 GB, Disk Space: 40 GB
Rancher First Node	CentOS 7.1 (64 bit)	Memory: 8 GB, Disk Space: 40 GB
Rancher Second Node	CentOS 7.1 (64 bit)	Memory: 8 GB, Disk Space: 40 GB
Rancher Third Node	CentOS 7.1 (64 bit)	Memory: 8 GB, Disk Space: 40 GB
AWX	CentOS 7.1 (64 bit)	Memory: 4 GB, Disk Space: 100 GB
Central	CentOS 7.1 (64 bit)	Memory: 4 GB, Disk Space: 64 GB
Pool Stage	CentOS 7.1 (64 bit)	Memory: 8 GB, Disk Space: 100 GB
Pool A	CentOS 7.1 (64 bit)	Memory: 16 GB, Disk Space: 128 GB
Pool B	CentOS 7.1 (64 bit)	Memory: 16 GB, Disk Space: 200 GB
Pool E	CentOS 7.1 (64 bit)	Memory: 16 GB, Disk Space: 128 GB
Pool G	CentOS 7.1 (64 bit)	Memory: 20 GB, Disk Space: 256 GB
Load balancer	CentOS 7.1 (64 bit)	Memory: 6 GB, Disk Space: 30 GB
Galera Cluster Node 1	CentOS 7.5 (64 bit)	Memory: 16 GB, Disk Space: 200 GB
Galera Cluster Node 2	CentOS 7.5 (64 bit)	Memory: 16 GB, Disk Space: 200 GB
Galera Cluster Node 3	CentOS 7.5 (64 bit)	Memory: 16 GB, Disk Space: 200 GB

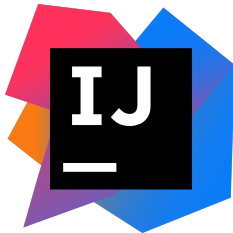
4.1.2 Software environment

4.1.2.1 PHPStorm



PHPStorm [8] is a PHP IDE that actually 'gets' the code. It supports PHP 5.3-8.2, provides on-the-fly error prevention, the best autocompletion and code refactoring, zero-configuration debugging, and an extended HTML, CSS, and JavaScript editor.

4.1.2.2 IntelliJ IDEA Ultimate



IntelliJ IDEA [9] is an integrated development environment written in Java for developing computer software written in Java, Kotlin, Groovy, and other JVM-based languages. It is developed by JetBrains and is available as an Apache 2 Licensed community edition, and in a proprietary commercial edition.

4.1.2.3 DataGrip



DataGrip [10] is the multi-engine database environment. We support MySQL, PostgreSQL, Microsoft SQL Server, Microsoft Azure, Oracle, Amazon Redshift, Sybase, DB2, SQLite, HyperSQL, Apache Derby and H2. If a DBMS has a JDBC driver, the connection to it is established via DataGrip.

4.1.2.4 Bash



Bash [11] is a Unix shell and command language written by Brian Fox for the GNU Project as a free software replacement for the Bourne shell.

4.1.2.5 YAML



YAML [12] YAML is a human-readable data-serialization language. It is commonly used for configuration files and in applications where data is being stored or transmitted.

4.1.2.6 Groovy



Groovy [13] Apache Groovy is a Java-syntax-compatible object-oriented programming language for the Java platform. It is both a static and dynamic language with features similar to those of Python, Ruby, and Smalltalk.

4.2 Used technologies

In this section, we describe used technologies to implement our DevOps pipelines according to DevOps practices.

4.2.1 Infrastructure As Code

Infrastructure as a code, briefly IAC [14], is a DevOps practice and implemented as a type of IT setup that can intervene to automatically manage and provision the technology stack for an application via code, in order to avoid the manual and interactive process. Indeed, several tools have been developed, such as Ansible

4.2.1.1 Ansible



Ansible [15] is a widely used automation tool that helps administrators to configure and manage servers easily. The tasks to be executed are defined in YAML format. These tasks can be grouped and executed in relation to other tasks using a YAML file called Playbook, which also defines variables and host lists.

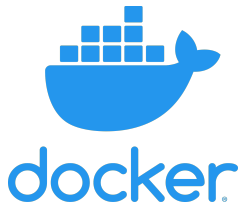
4.2.1.2 AWX Server



AWX [16] is a user-friendly open-source web-based tool that allows for the easy management of Ansible. It provides a user interface, REST API, and job engine for Ansible, making it simpler to manage. With AWX, it is possible to manage Playbooks, inventories, and schedule jobs to run.

4.2.2 Continuous deployment and Containerization

4.2.2.1 Docker



Docker [17] is a popular platform for developing, shipping, and running applications using container technology. It provides a lightweight, portable, and scalable environment that isolates applications from the underlying host system. Docker allows developers to package an application with its dependencies into a single unit called a container, making it easier to deploy and manage.

4.2.2.2 Container orchestration with Kubernetes



Container orchestration is the process of automating the management, scheduling, and scaling of individual containers to support the lifecycles of containerized applications within dynamic environments. Kubernetes [18], an open-source platform, well-suited for container orchestration by providing fault-tolerant and highly available environments for running applications at scale. It automates essential tasks such as scheduling, networking, and resource management, making it easier to deploy and manage containerized applications efficiently.

4.2.2.3 Rancher Server



Rancher [19] is a software tool that helps manage and deploy containerized applications across multiple servers. It simplifies the process of setting up and managing a container orchestration platform, such as Kubernetes, by providing an intuitive user interface and automation features. With Rancher, users can easily scale, monitor, and upgrade their containerized applications, making it a valuable tool for container management in complex environments.

4.2.3 Continuous integration and Version control

4.2.3.1 Beanstalk



Beanstalk [20] is a complete workflow platform to write, review, deploy code, and allows to keep the code in a Git or Subversion (SVN) repository, to perform code reviews with peers to write higher quality and bug-free code, and to deploy the code from Beanstalk to servers.

4.2.3.2 Jenkins



Jenkins

Jenkins [21] Jenkins is a self-contained, open source automation server which can be used to automate all sorts of tasks related to building, testing, and delivering or deploying software. Jenkins can be installed through native system packages, Docker, or even run standalone by any machine with a Java Runtime Environment (JRE) installed.

4.2.4 Continuous testing

4.2.4.1 Snyk



Snyk [22] is a software security platform that helps developers to find and fix vulnerabilities in their open-source libraries and container images. It provides real-time monitoring, automated vulnerability scanning, and actionable insights to enhance the security of software applications. With its easy integration into the development workflow, Snyk empowers developers to proactively identify and remediate security issues, ensuring the integrity and safety of their code.

4.2.4.2 PHPStan



PhpStan [23] is a static analysis tool for PHP code that performs a comprehensive code analysis to detect potential errors and improve code quality. It analyzes the codebase and provides detailed feedback on type errors, undefined variables, unused code, and other common mistakes. By leveraging static analysis, PhpStan helps developers identify and fix issues early in the development process, leading to more reliable and maintainable PHP applications.

4.2.5 Database management

4.2.5.1 MariaDB



MariaDB [24] is a free and open-source database software that helps organize and store structured data. It is similar to MySQL and can be used as a replacement for it. With MariaDB, you can manage and retrieve data efficiently and reliably. It offers additional features like replication and high availability, making it a popular option for different types of applications and websites.

4.2.5.2 Galera Cluster



Galera Cluster [25] is a synchronous multi-master database replication solution for high availability and scalability. It enables real-time data replication across multiple nodes, allowing them to work as a single database cluster. Galera Cluster provides automatic node recovery, data consistency, and distributed transaction support, making it suitable for applications that require high availability and fault tolerance.

4.2.6 Continuous monitoring

4.2.6.1 ELK Stack

Elasticsearch



Elasticsearch [26] is a distributed, real-time search and analytics engine. It is designed to store, search, and analyze large volumes of data in near real-time. Elasticsearch is built on top of the Apache Lucene library and provides a RESTful API for indexing and searching data, making it widely used for log analytics, full-text search, and data exploration.

Logstash



Logstash [26] is an open-source data processing pipeline that allows to collect, transform, and ingest data from various sources into a centralized repository. It provides a wide range of input plugins to gather data from different systems, filter plugins to process and enrich the data, and output plugins to send the processed data to various destinations, such as Elasticsearch. Logstash is often used

Kibana



as a log shipper and data pipeline for aggregating and parsing logs. Kibana [26] is an open-source data visualization and exploration tool that works in conjunction with Elasticsearch. It provides a web-based interface for querying, analyzing, and visualizing data stored in Elasticsearch. Kibana offers a variety of features, including dashboards, charts, maps, and graphs, to help users gain insights from their data. It is commonly used for log monitoring, metrics analysis, and creating custom visualizations to support business intelligence and operational monitoring.

4.2.6.2 TIG Stack

Telegraf



Telegraf [27] is an open-source agent that collects, processes, and sends metrics and data from various sources to a time-series database. It is designed to gather data from systems, services, and sensors, providing a flexible and extensible plugin architecture for data collection. Telegraf supports a wide range of input plugins to gather metrics from different sources and output plugins to send the data to various destinations, including InfluxDB.

InfluxDB



InfluxDB [27] is an open-source time-series database built to handle high volumes of time-stamped data. It is optimized for storing, querying, and visualizing time-series data, making it suitable for storing metrics, events, and sensor data. InfluxDB uses a flexible data model with measurements, tags, and fields, and supports a SQL-like query language (InfluxQL) for data retrieval and analysis. It provides high-performance data ingestion and scalability, making it well-suited for real-time monitoring and analytics.

Grafana



Grafana [27] is an open-source data visualization and analytics platform. It allows users to create and display interactive dashboards, charts, and graphs from various data sources, including InfluxDB. Grafana supports a wide range of data visualization options, customizable panels, and alerting capabilities. It is commonly used for monitoring and observability, providing real-time insights and visualizations for metrics, logs, and other data sources.

4.2.7 Networking and Collaboration

4.2.7.1 Webhook



A webhook [28] is a way for two different software applications to communicate with each other by sending a small message when an event occurs. It allows one application to trigger an action in another application, without the need for constant manual intervention.

4.2.7.2 HAProxy



HAProxy [29] is a software that functions as a load balancer and proxy server. It distributes incoming network traffic across multiple servers to improve performance and reliability. HAProxy acts as an intermediary between clients and servers, optimizing the traffic flow and ensuring efficient handling of requests. With its high availability and advanced load balancing capabilities, HAProxy is commonly used to improve the scalability and resilience of web applications.

4.2.7.3 Discord



Discord [30] is a versatile communication platform that allows users to create communities, engage in voice and text conversations, and share media content. It offers real-time, multi-user interactions and features such as voice and video calling, direct messaging, and customizable channels. Widely used by gamers and other communities, Discord provides a user-friendly and customizable environment for connecting, collaborating, and engaging in real-time conversations.

4.3 Physical architecture

To develop our solution, we used previously described technologies in order to ensure maximum availability and scalability. Indeed, our software physical architecture is illustrated via a deployment diagram shown in figure 4.1.

This diagram contains a set of software and hardware components that work together to maintain a highly available and scalable system. The components include servers such as the Jenkins build node, the Grafana server, InfluxDB server, ELK server, Rancher cluster, AWX server, and Galera cluster nodes. These servers are responsible for various tasks such as build automation, monitoring, logging, container orchestration, and database clustering.

The deployment diagram depicts the relationships between these components, showing how they are interconnected and communicate with each other.

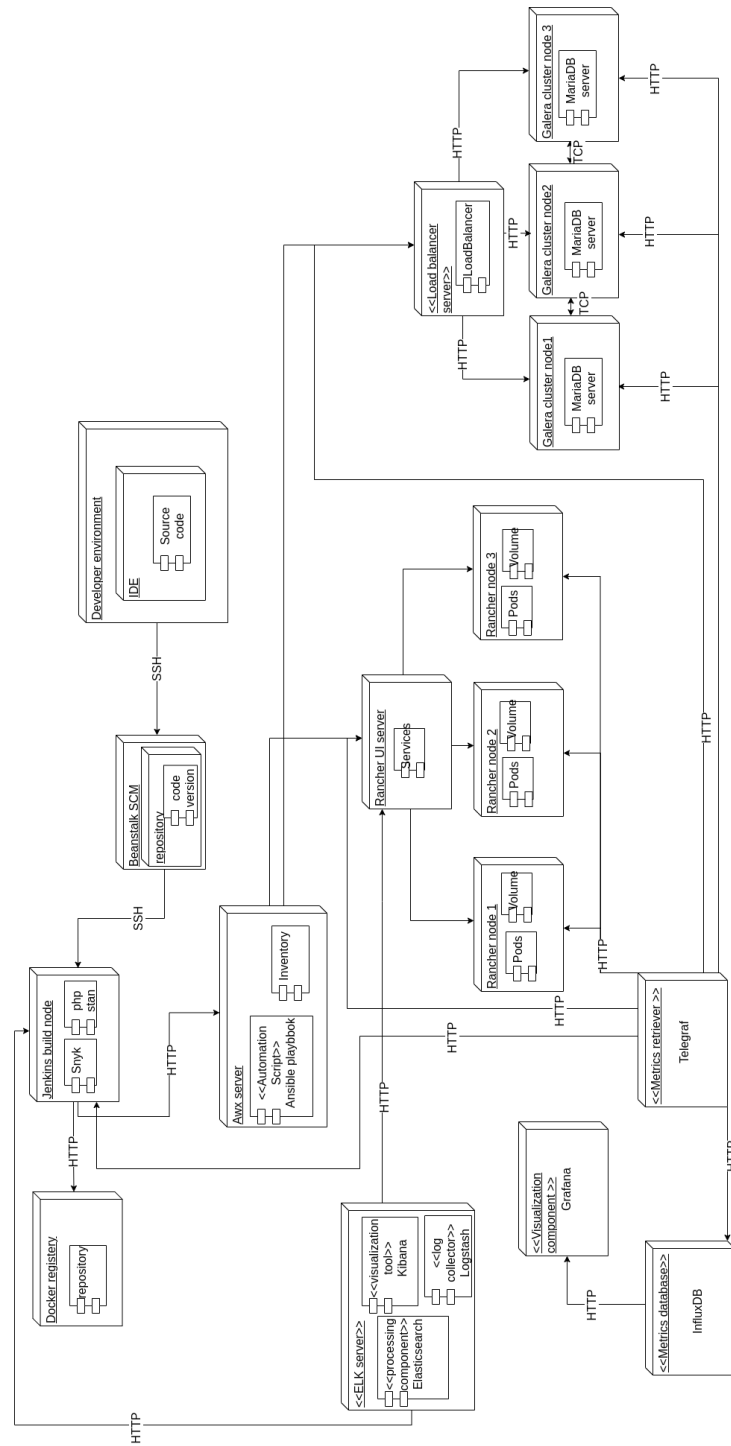


Figure 4.1: Global deployment diagram

4.4 Proof of concept

4.4.1 Kanban implementation

Figure 4.2 displays the Kanban board using ClickUp tool.

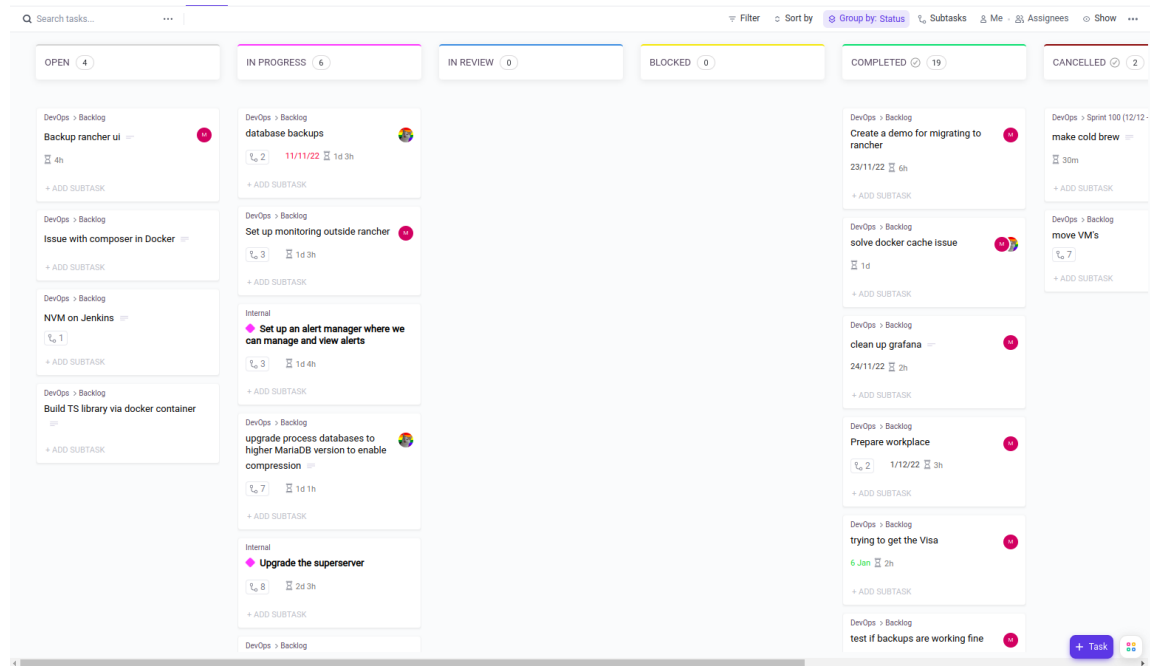


Figure 4.2: Kanban Board (ClickUp tool)

Figure 4.3 illustrates the task of deploying the ELK stack, including various subtasks involved in the process.

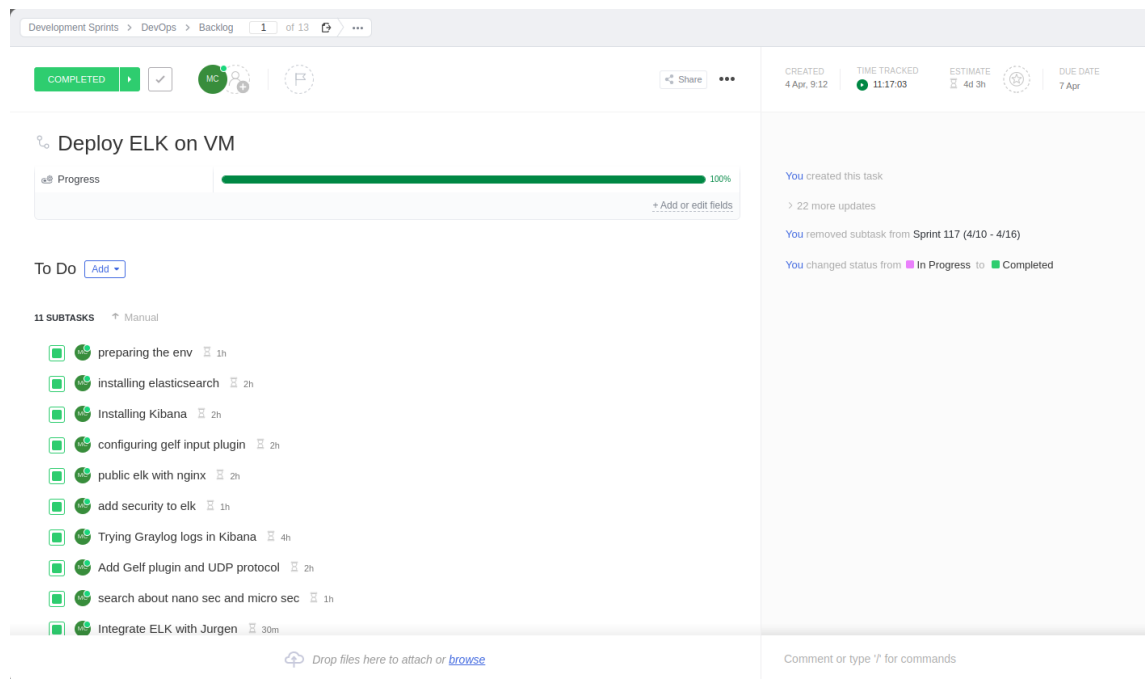


Figure 4.3: Elk deployment task

The figure 4.4 illustrates the automated process of fixing the database cluster, including its subtasks.

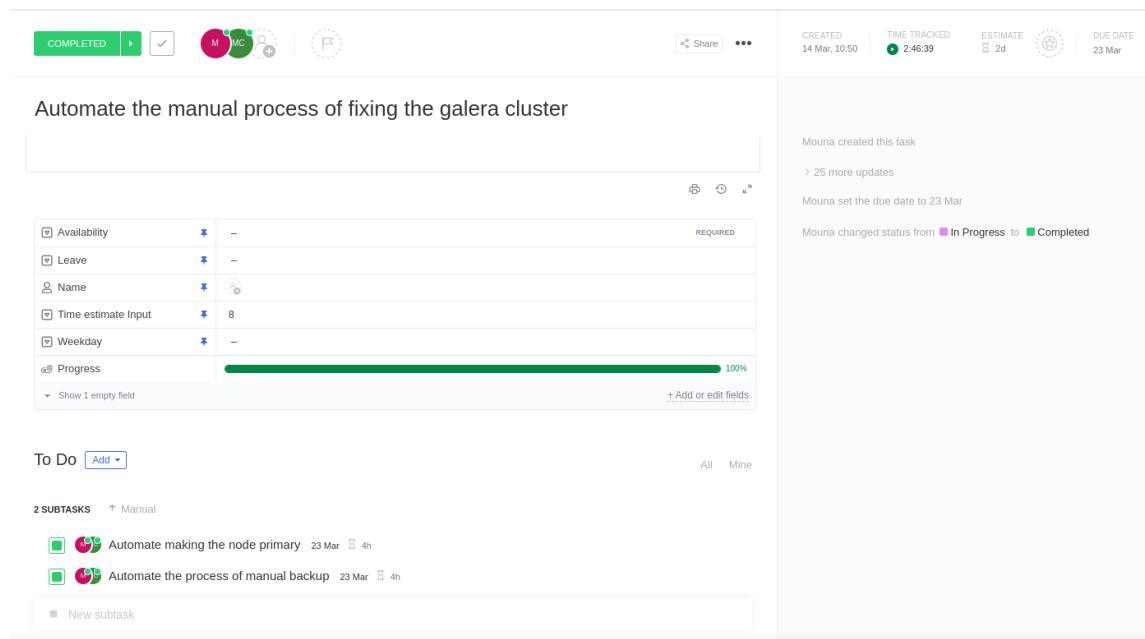


Figure 4.4: Automate fix for galera cluster task

4.4.2 CI/CD pipeline

We describe briefly our CI/CD pipeline stages in figure 4.5 and real-time execution in figure 4.9.

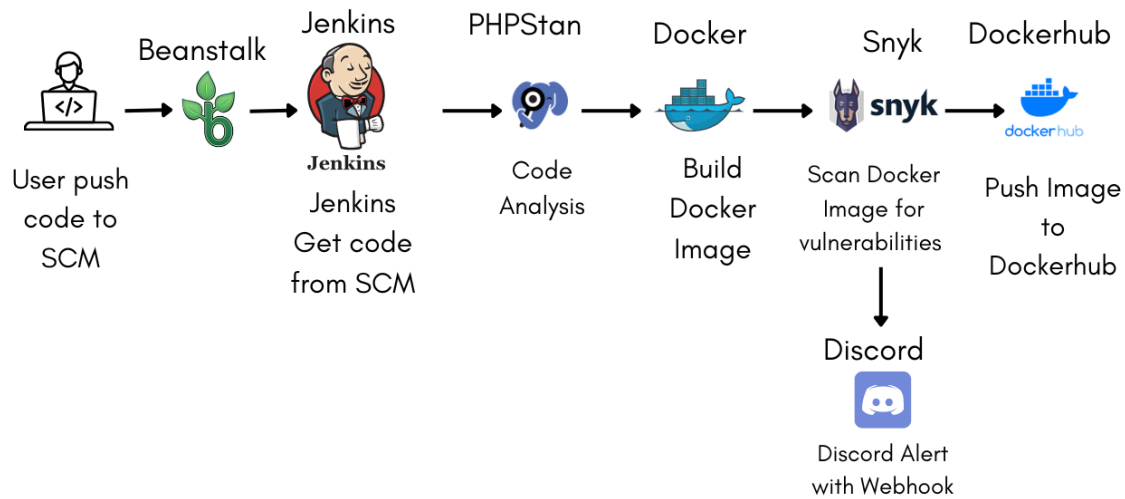


Figure 4.5: CI/CD Pipeline

- **Cloning the Code from the BeanstalkApp Repository (From any branch except Main and Dev)**

In this stage, the code is cloned from our BeanstalkApp repository. The source code is cloned from the branch that triggered the pipeline by running the `jenkinsfile`.

- **Code analysis with PHPstan**

In this stage, the code is analyzed using a tool called `phpstan`. It helps identifying potential errors, bugs, and code quality issues in PHP code as shown in figure 4.6. It helps ensure that the code follows best practices and is free from common mistakes. When PHP Stan detects a bug or when the tests fail, it halts the subsequent steps in the pipeline. This is a crucial point where developers are required to address the issues by updating the code.

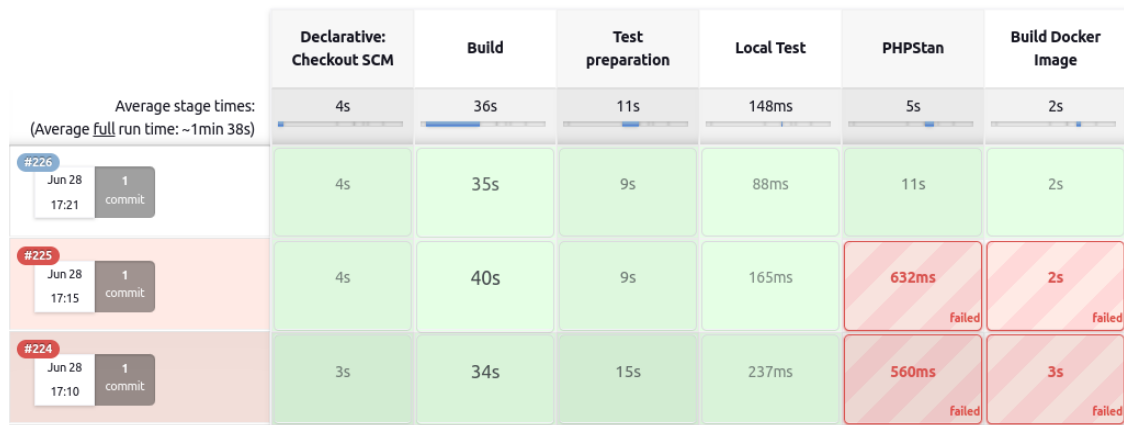


Figure 4.6: PHPStan real-time error detection

- **Build Docker image**

Here, a Docker image is created, which is a lightweight, standalone executable package that contains all the necessary components to run the application. The Docker image encapsulates the application code, dependencies, and runtime environment. It allows for easy deployment and consistent execution across different environments.

- **Scanning vulnerabilities with Snyk**

In this stage, we use Snyk to scan the Docker image for any security vulnerabilities. Snyk analyzes the image's components, including the base image, operating system, libraries, and other dependencies, to identify known vulnerabilities as shown in figure 4.7. This helps us to identify and address potential security issues before deploying the image. Snyk plays a crucial role by identifying a critical vulnerability in the base image. As a result, it halts the subsequent step of pushing the image to prevent the deployment of potentially insecure software.

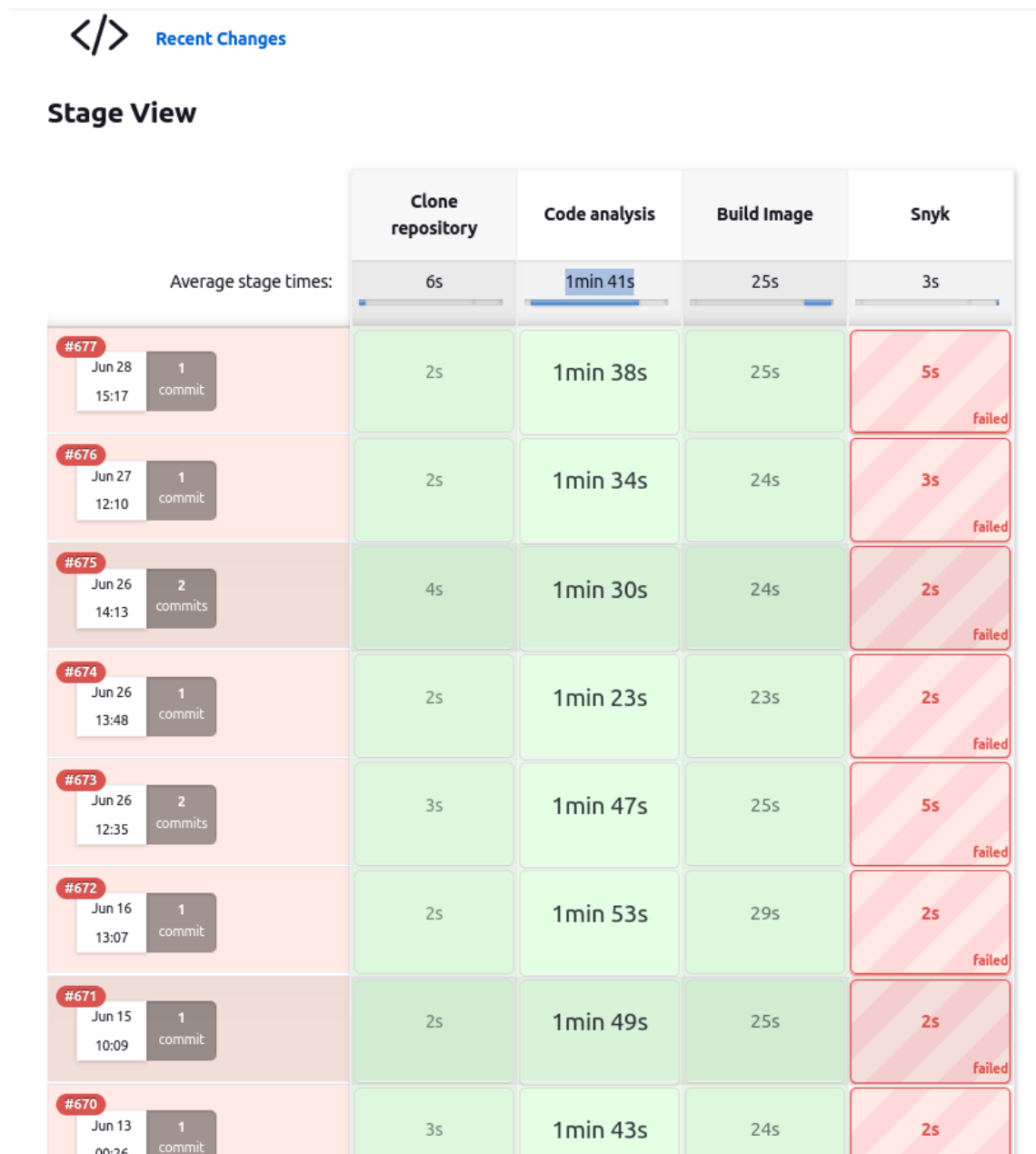


Figure 4.7: Snyk real-time vulnerabilities detection

- Push image to Docker Hub (if vulnerabilities are okay):

If the vulnerability scan performed by Snyk doesn't find any issues in the Docker image, the image is pushed to the Docker Hub. This involves pushing the image to the Docker Hub repository, making it available for others to access and use.

- **Notify via Discord (if vulnerabilities are found):**

If vulnerabilities are found during the Snyk scan, a notification is sent via Discord to inform developers about the identified vulnerabilities. This notification allows the appropriate actions to be taken to address and resolve the security issues before proceeding with pushing the image to Docker Hub (Figure 4.8).

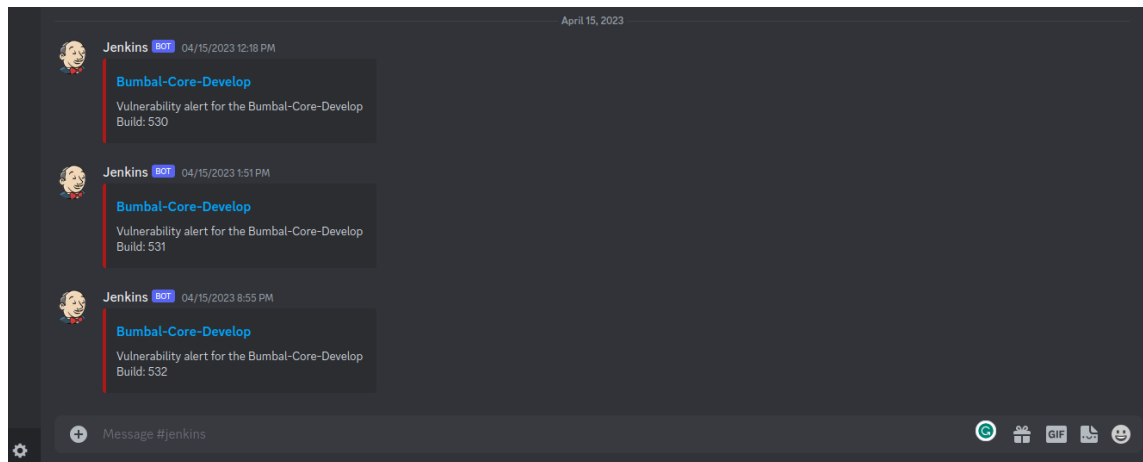


Figure 4.8: Development team notification

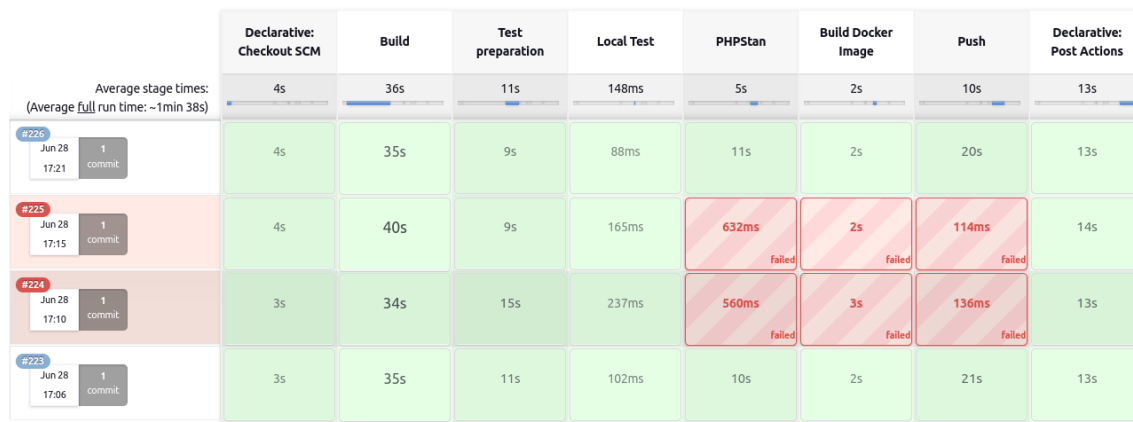
Stage View

Figure 4.9: CI/CD pipeline real-time execution visualisation

Triggers

Using a Webhook created in SCM , to trigger the Continuous Integration when there's a push on any branch except 'main' and 'dev' branches Using a Webhook to trigger the Release Pipeline/Continuous Deployment when we push on 'main' branch. The figures 4.10 and 4.11 show the steps to create and configure the Integration Webhook.

Dashboard > Continuous_Integration >

General Build Triggers Advanced Project Options Pipeline

Build Triggers

- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☒ Build when a change is pushed to GitLab. GitLab webhook URL: http://localhost:8080/project/Continuous_Integration ?

Enabled GitLab triggers

- ☒ Push Events
 - ☐ Push Events in case of branch delete
 - ☐ Opened Merge Request Events
 - ☐ Build only if new commits were pushed to Merge Request ?
 - ☐ Accepted Merge Request Events
 - ☐ Closed Merge Request Events

Rebuild open Merge Requests

Never

☐ Approved Merge Requests (EE-only)

Figure 4.10: Continuous Integration Trigger Configuration

Allowed branches

- ☐ Allow all branches to trigger this job ?
- ☐ Filter branches by name ?
- ☒ Filter branches by regex ?

Source Branch Regex

^((?!.*main.*)(?!.*dev.*))*\$

Target Branch Regex

☐ Filter merge request by label

Secret token ?

f4d163f85f854678c6f6a79e3bb46d4a

Generate

Figure 4.11: Trigger Branch Configuration

4.4.3 Containerization

4.4.3.1 Containers Orchestrated by Rancher

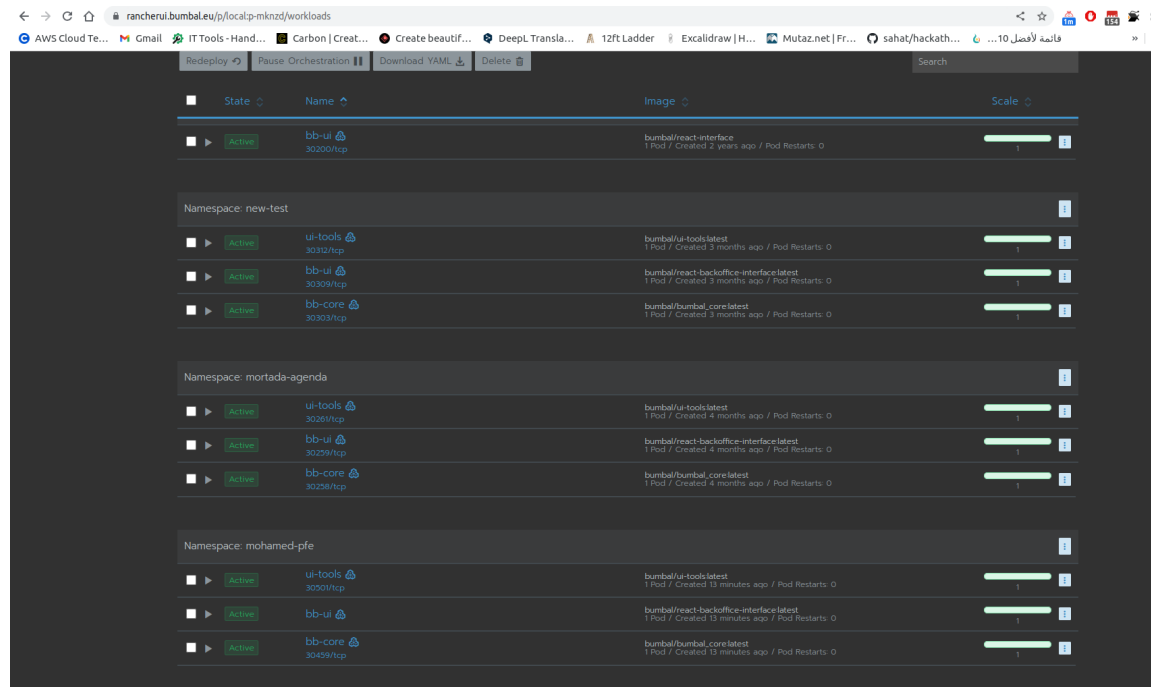


Figure 4.12: Rancher’s running containers interface

Figure 4.12 showcases the successful completion of the instance creation process, as indicated by the appearance of a new namespace in Rancher. This signifies that the instance has been successfully deployed and is now ready for use within the designated namespace.

4.4.4 Continuous monitoring and automated problem fixing

4.4.4.1 Checking logs with Kibana

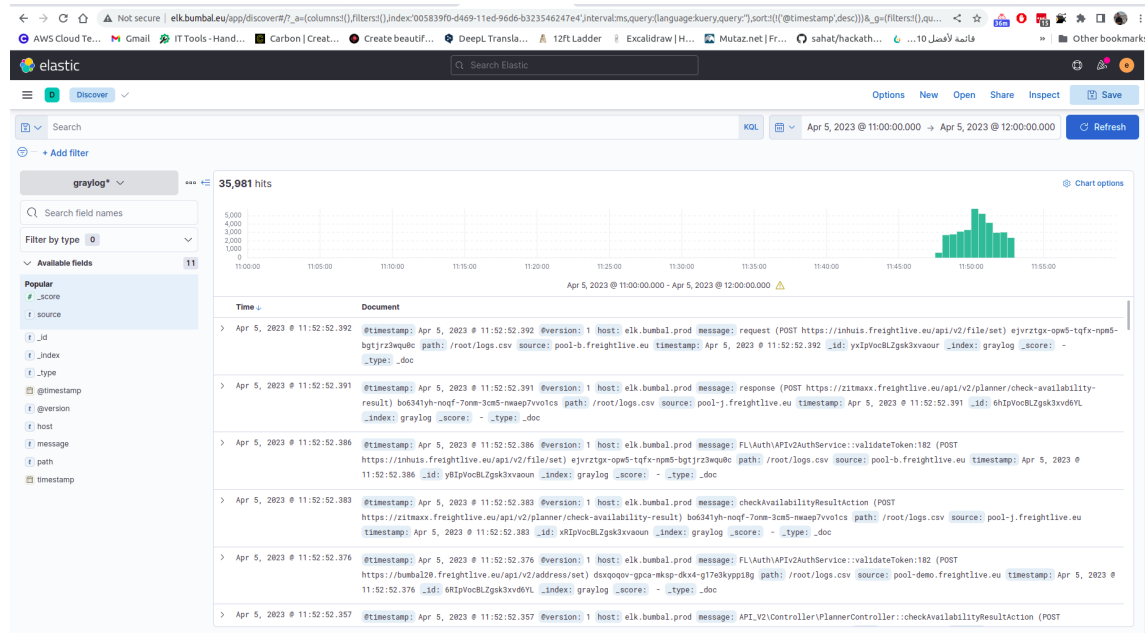


Figure 4.13: Kibana interface

we can observe the process of checking logs using Kibana, which provides a user-friendly interface for efficient log analysis as shown in figure 4.13

4.4.4.2 Monitoring environment resources usage

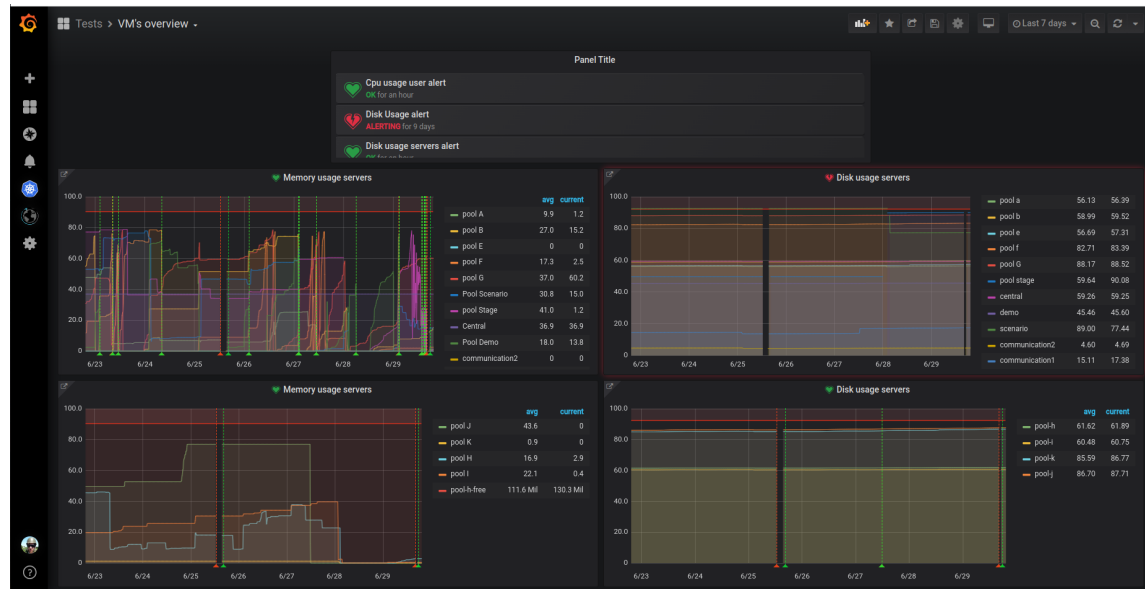


Figure 4.14: Resources metrics interface in Grafana

Figure 4.14 displays the Grafana dashboard, which offers a comprehensive view of resource monitoring, enabling users to track and analyze system resources in real time.

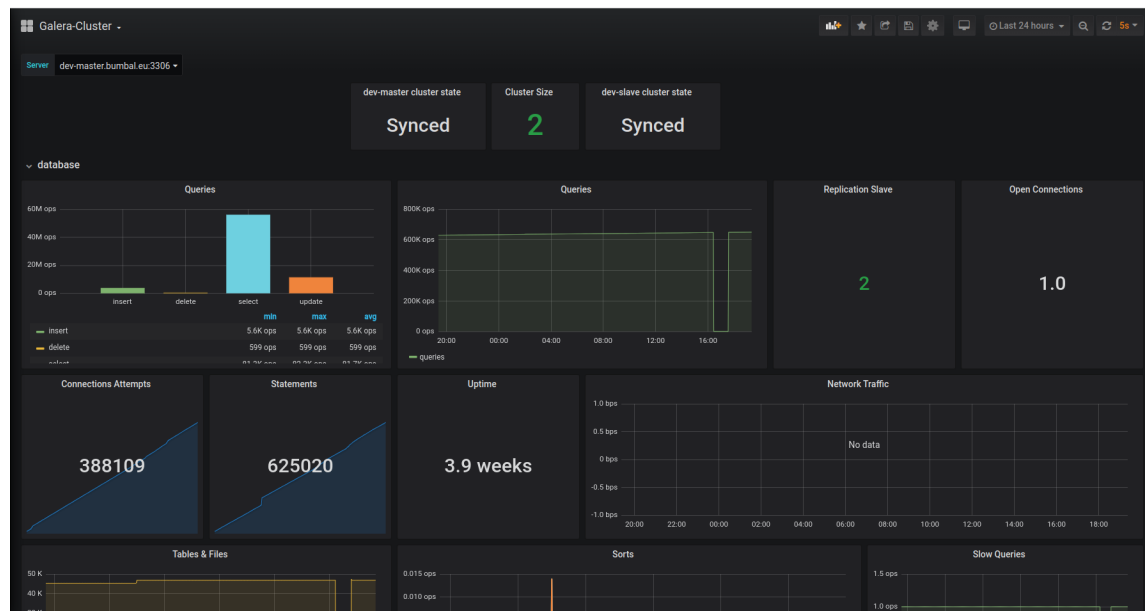


Figure 4.15: Galera Cluster real-time metrics

Figure 4.15 illustrates Grafana dashboards presenting performance metrics for monitoring the Galera cluster, providing valuable insights into its performance

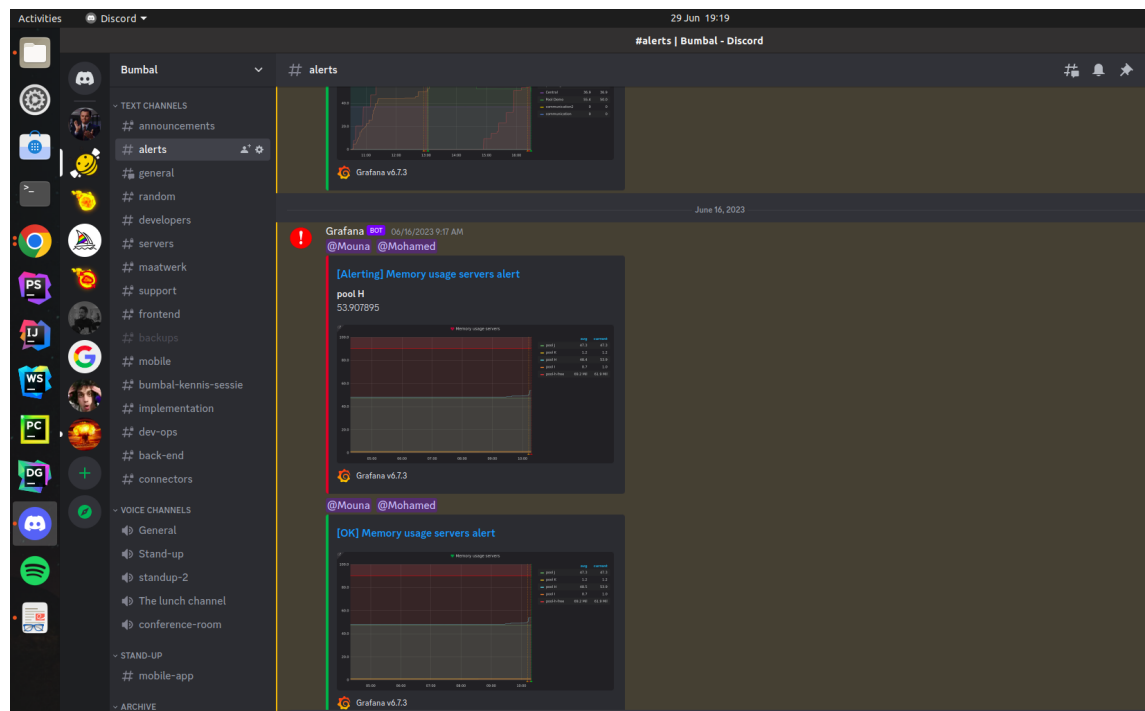


Figure 4.16: Automated fixed resources problem

Figure 4.16 showcases an alert indicating resource problems and the subsequent automated resolution process, ensuring efficient automated management and resolution of resource issues.

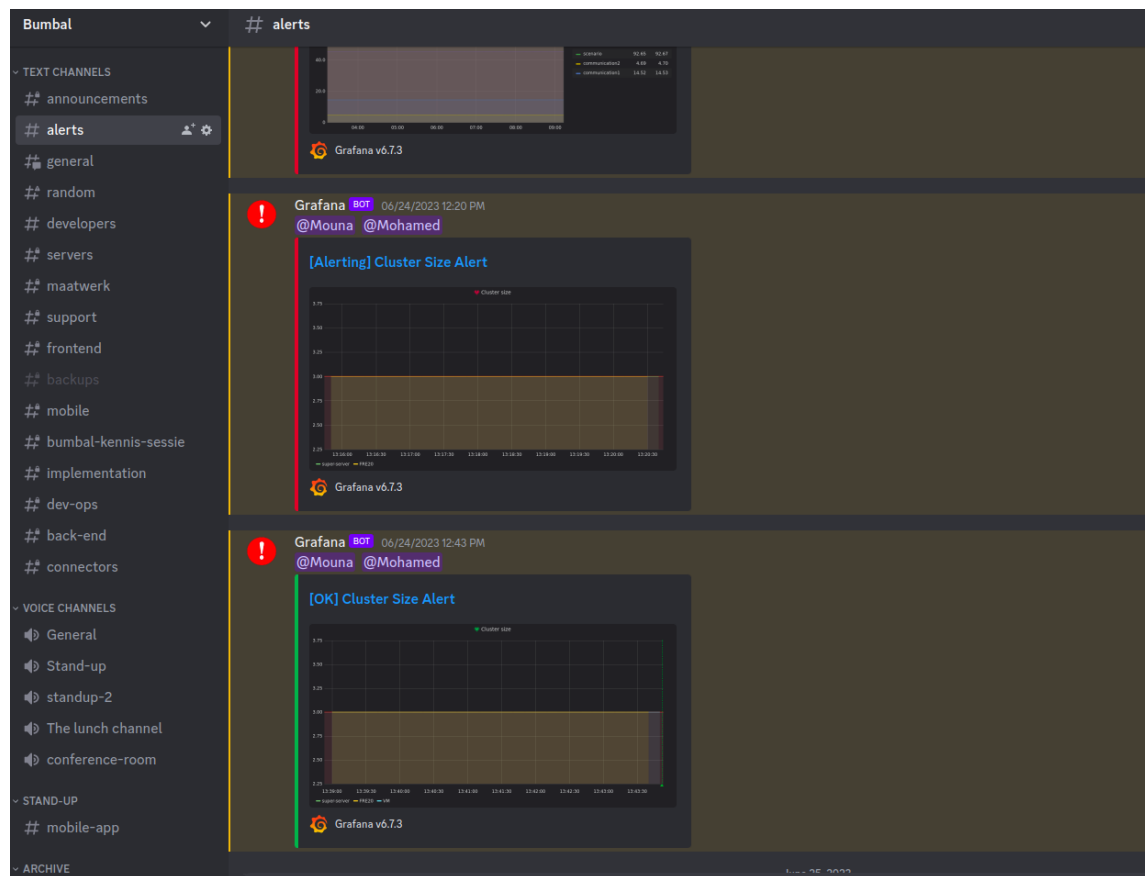


Figure 4.17: Self-fixed incident in DB cluster

Figure 4.17 demonstrates a self-fixed incident in the DB cluster, where an alert was triggered, and the system automatically resolved the issue without the need for manual intervention, ensuring continuous availability and stability of the database cluster.

4.4.5 DevOps engineer administration

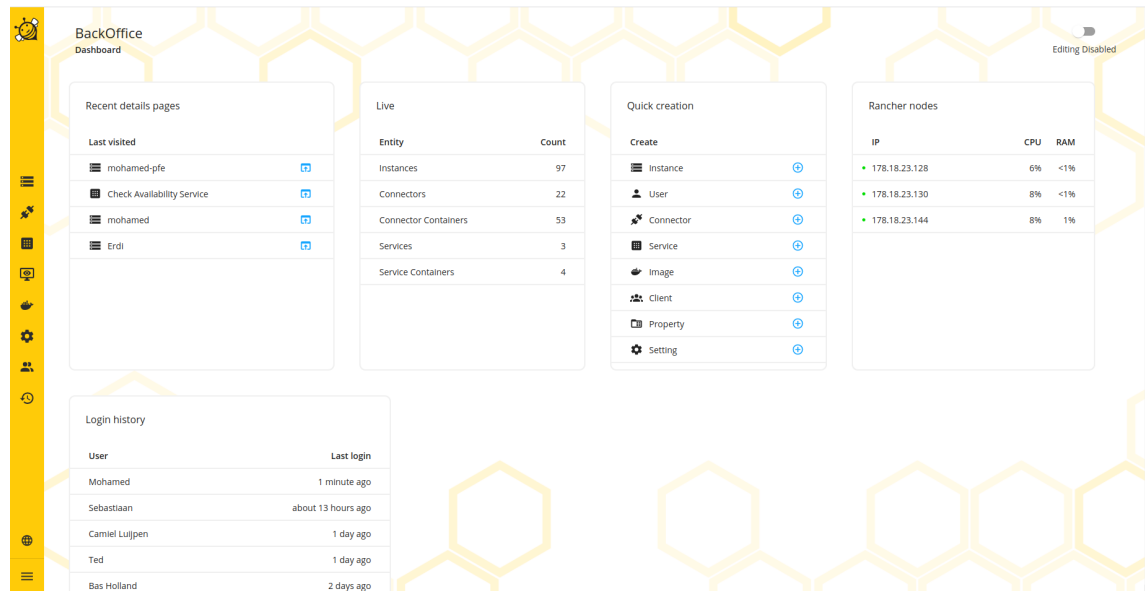


Figure 4.18: Backoffice home page interface

The figure 4.18 represents the backoffice's home page, offering a comprehensive interface for users to efficiently manage various system functionalities such as viewing a list of images, managing tokens, checking the summary of cluster nodes health status, quickly creating instances, services, and connectors, as well as reviewing login history.

4.4.5.1 Create an app instance

Create Instance

Name
mohamed-pfe

Domain
mohamed-pfe .bun

Admin Email
admin@mohamed-pfe.bumbal.eu

Admin Password
e1bd60a5-8a72-57ef-70a3-bbf0cf585739

☐ Mark Instance For Migration

☐ VPS Already Has An Existing Freightlive.Eu Environment

Advanced Options

UI Image

UI Image Version

Core Image

Core Image Version

Tools Image Version

☐ Launch Instance On Stage Servers

SAVE

Figure 4.19: Create an instance interface

Figure 4.19 demonstrates how users can easily create new instances which is a customized app from the Bumbal product for the company's customers.

4.4.5.2 Running instance interface

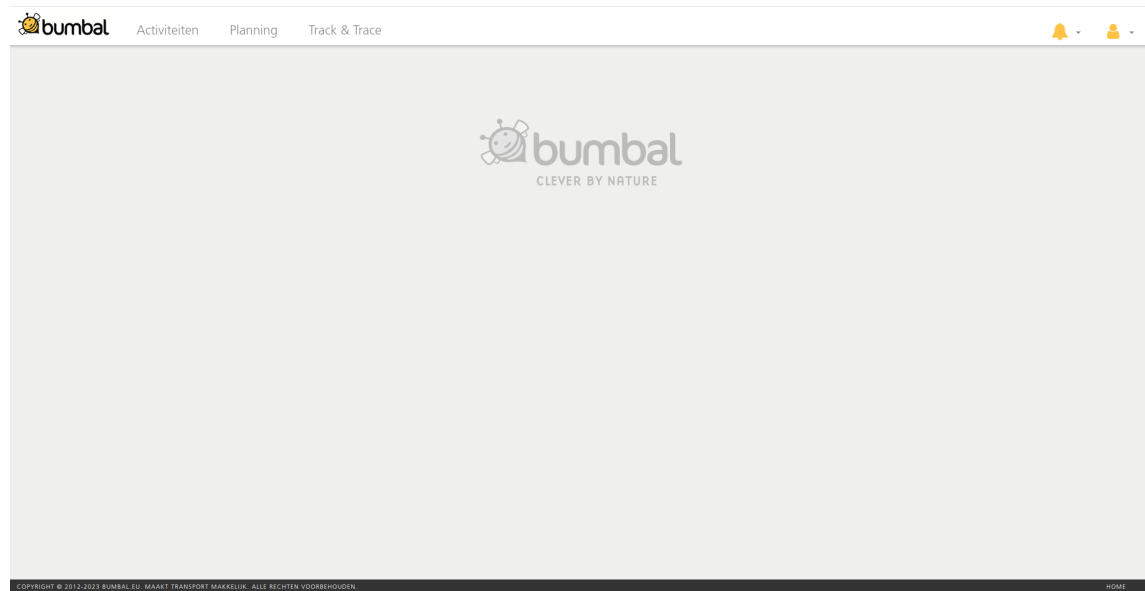
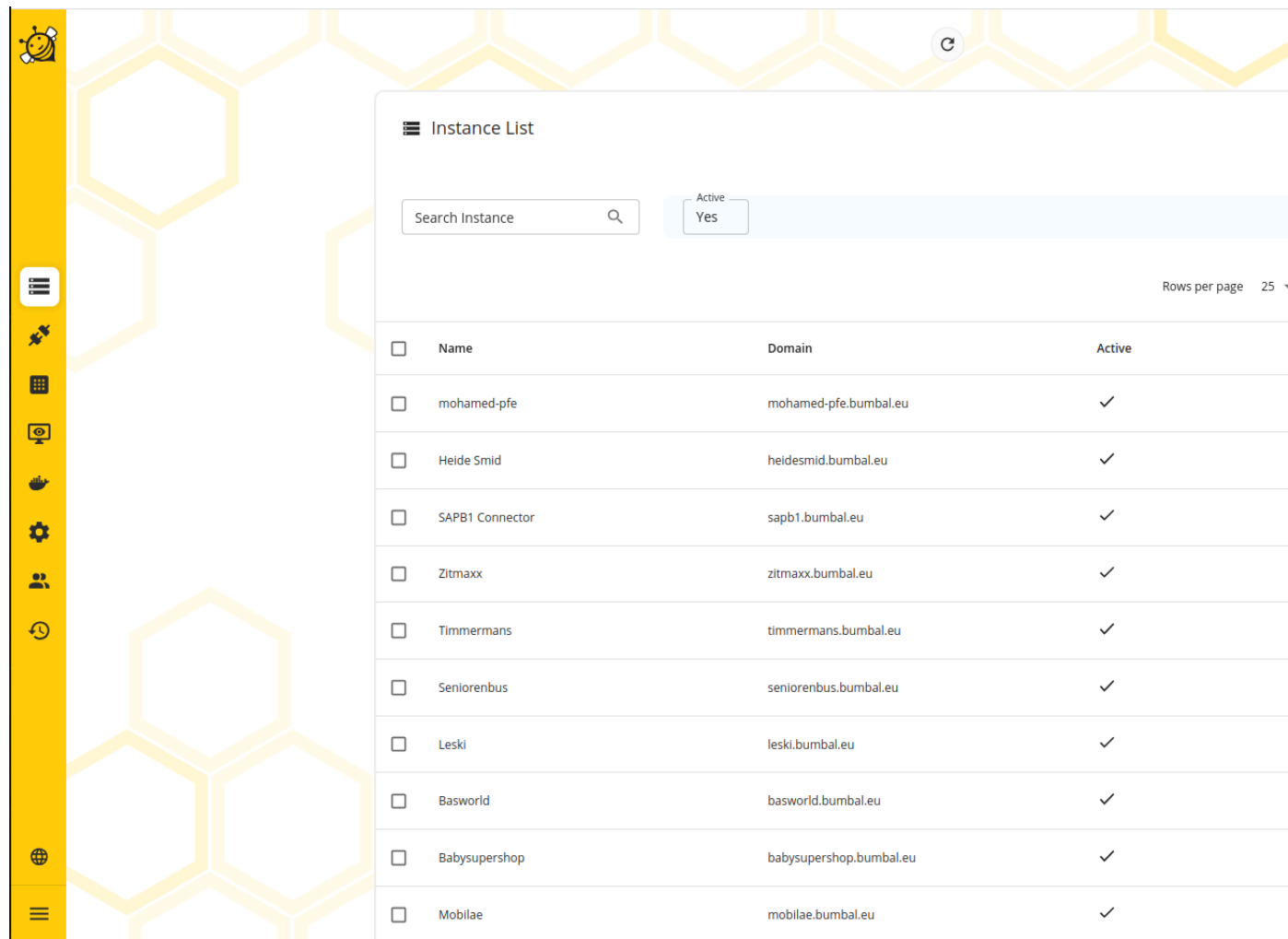


Figure 4.20: Bumbal default instance interface

Figure 4.20 represents an active running instance that is fully operational and ready to be used by customers. This signifies the successful deployment and configuration of the instance, making it available for users to access and using its functionalities.

4.4.5.3 List of instances in backoffice



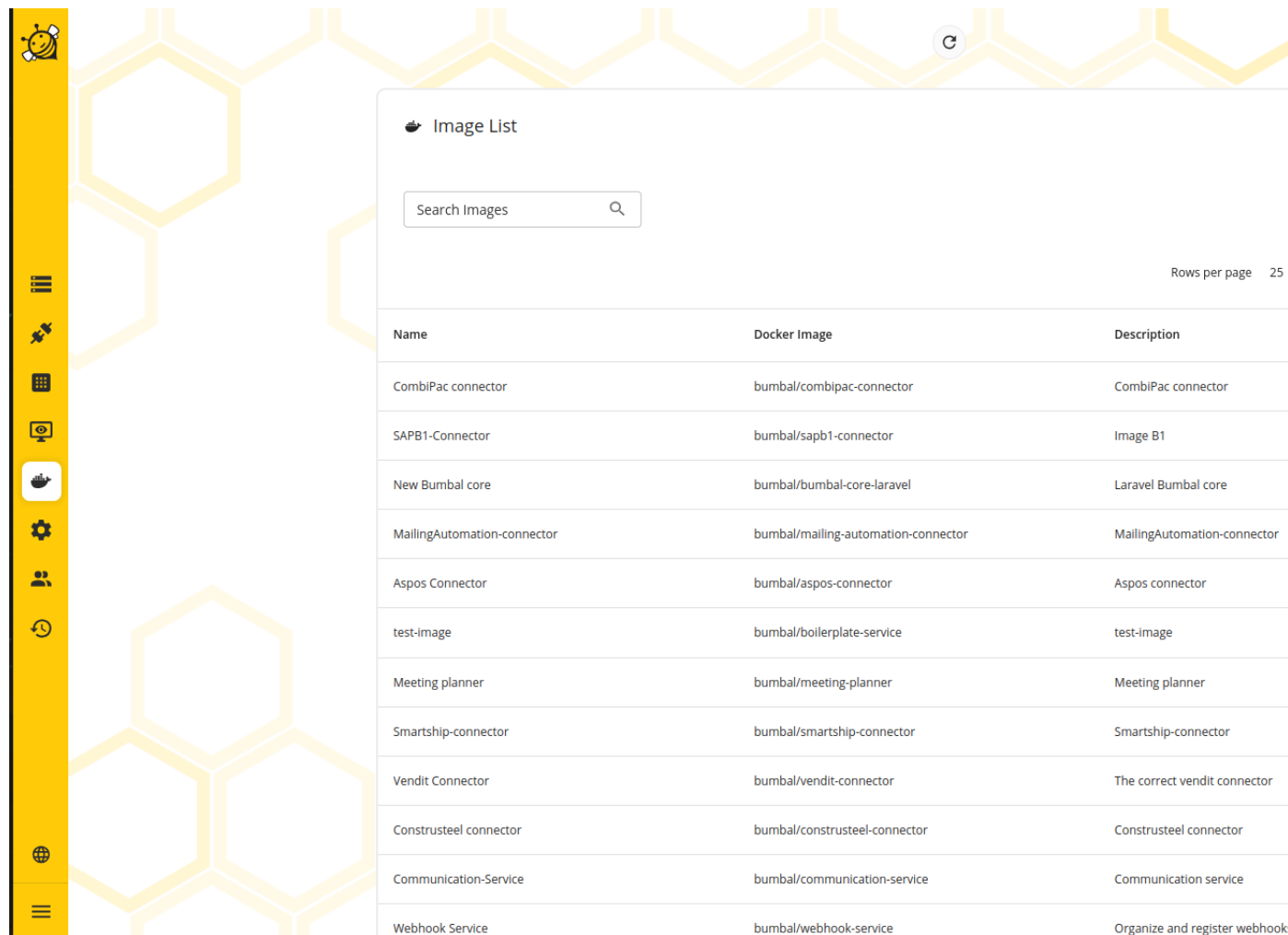
The screenshot shows a web application interface with a yellow sidebar on the left containing various icons. The main content area displays a table titled "Instance List". Above the table, there is a search bar labeled "Search Instance" and a filter button labeled "Active Yes". The table has columns for "Name", "Domain", and "Active". The "Active" column contains checkmarks for all listed instances. The table is paginated, showing 10 rows out of 25.

	Name	Domain	Active
☐	mohamed-pfe	mohamed-pfe.bumbal.eu	✓
☐	Heide Smid	heidesmid.bumbal.eu	✓
☐	SAPB1 Connector	sapb1.bumbal.eu	✓
☐	Zitmaxx	zitmaxx.bumbal.eu	✓
☐	Timmermans	timmermans.bumbal.eu	✓
☐	Seniorenbus	seniorenbus.bumbal.eu	✓
☐	Leski	leski.bumbal.eu	✓
☐	Basworld	basworld.bumbal.eu	✓
☐	Babysupershop	babysupershop.bumbal.eu	✓
☐	Mobilae	mobilae.bumbal.eu	✓

Figure 4.21: List of instances in the backoffice interface

Figure 4.21 showcases a list of instances, providing an overview of the existing deployments and their respective details in a concise and organized manner.

4.4.5.4 Docker images list



Name	Docker Image	Description
CombiPac connector	bumbal/combipac-connector	CombiPac connector
SAPB1-Connector	bumbal/sapb1-connector	Image B1
New Bumbal core	bumbal/bumbal-core-laravel	Laravel Bumbal core
MailingAutomation-connector	bumbal/mailling-automation-connector	MailingAutomation-connector
Aspos Connector	bumbal/aspos-connector	Aspos connector
test-image	bumbal/boilerplate-service	test-image
Meeting planner	bumbal/meeting-planner	Meeting planner
Smartship-connector	bumbal/smartship-connector	Smartship-connector
Vendit Connector	bumbal/vendit-connector	The correct vendit connector
Construsteel connector	bumbal/construsteel-connector	Construsteel connector
Communication-Service	bumbal/communication-service	Communication service
Webhook Service	bumbal/webhook-service	Organize and register webhooks

Figure 4.22: Docker images list in the backoffice interface

Figure 4.22 displays a comprehensive list of Docker images available for future use, providing a convenient reference for selecting and deploying containers as needed.

4.4.5.5 List of services in the backoffice

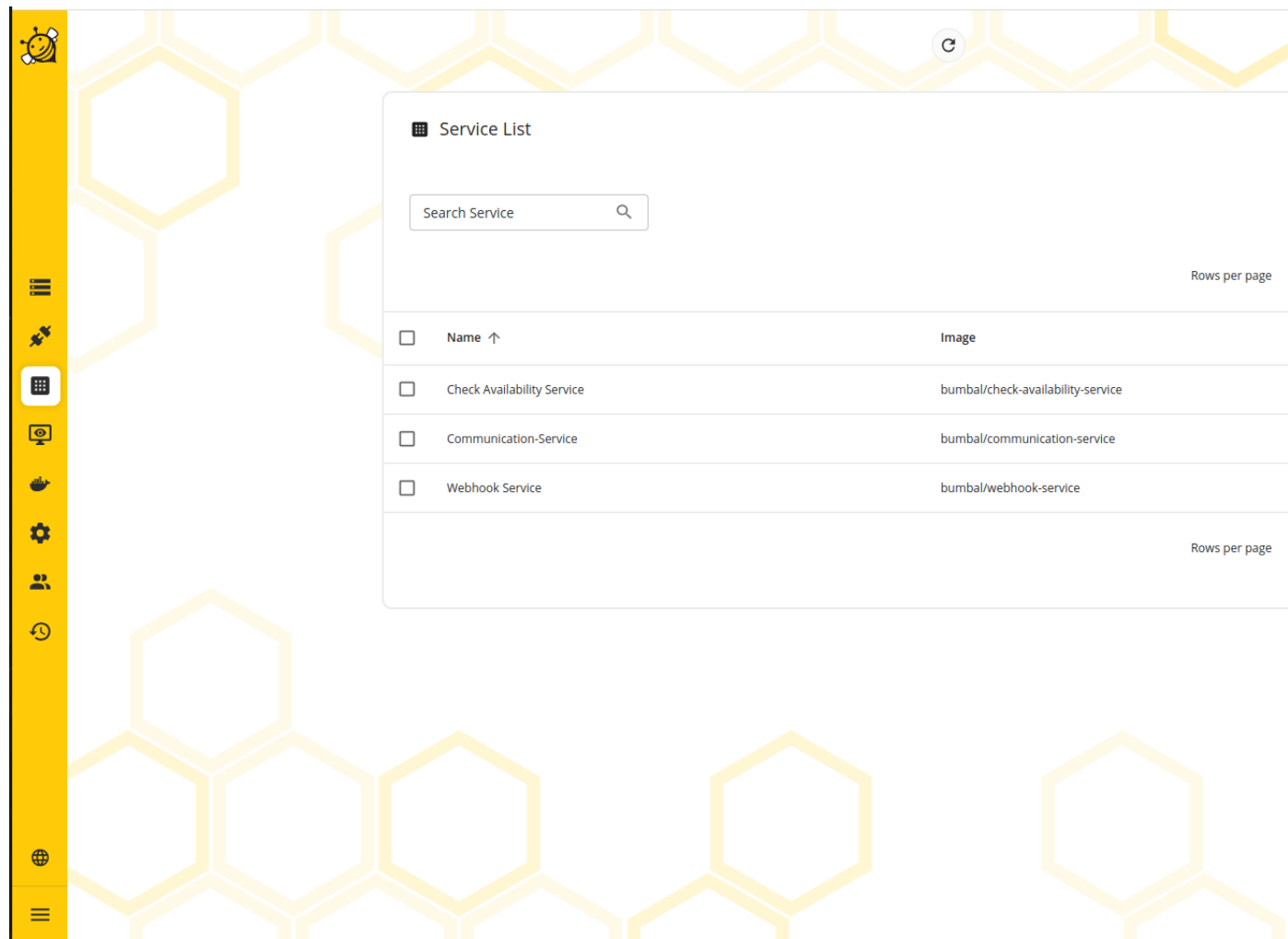
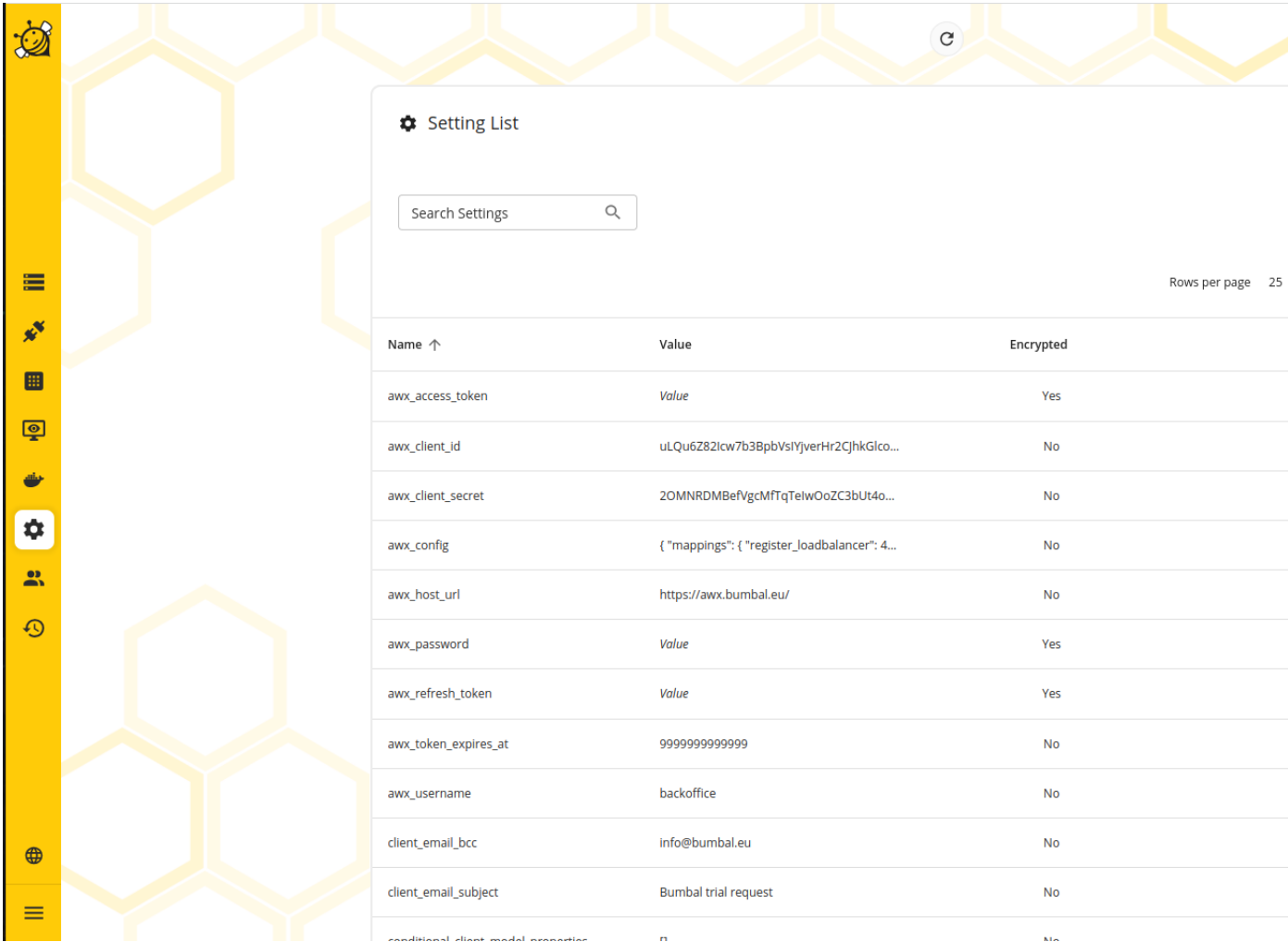


Figure 4.23: Service list interface

Figure 4.23 displays a comprehensive list of services, providing an overview of the various deployed services and their associated information.

4.4.5.6 List of token used in backoffice

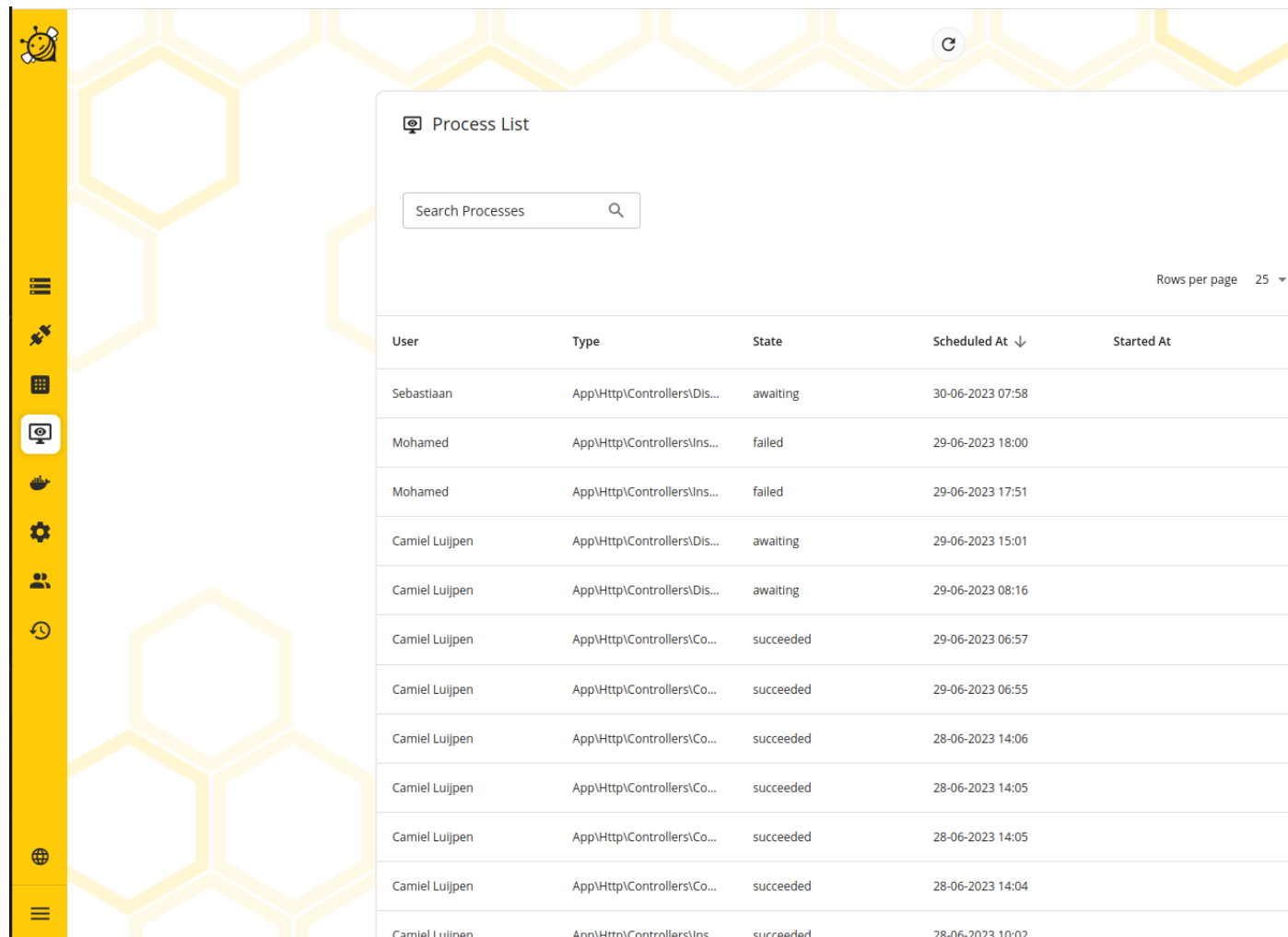


Setting List		
Search Settings		
Rows per page 25		
Name ↑	Value	Encrypted
awx_access_token	Value	Yes
awx_client_id	uLQu6Z82lcw7b3BpbVslYjverHr2CJhkGlco...	No
awx_client_secret	2OMNRDMBefVgcMftqTelwOoZC3bUt4o...	No
awx_config	{ "mappings": { "register_loadbalancer": 4...	No
awx_host_url	https://awx.bumbal.eu/	No
awx_password	Value	Yes
awx_refresh_token	Value	Yes
awx_token_expires_at	9999999999999	No
awx_username	backoffice	No
client_email_bcc	info@bumbal.eu	No
client_email_subject	Bumbal trial request	No
conditional_client_model_registration	n	No

Figure 4.24: List of credentials and tokens interface

Figure 4.24 showcases a detailed list of credentials and tokens, which serve as secure authentication mechanisms for accessing various systems and services.

4.4.5.7 List of processes in the backoffice



User	Type	State	Scheduled At ↓	Started At
Sebastiaan	App\Http\Controllers\Dis...	awaiting	30-06-2023 07:58	
Mohamed	App\Http\Controllers\Ins...	failed	29-06-2023 18:00	
Mohamed	App\Http\Controllers\Ins...	failed	29-06-2023 17:51	
Camiel Luijpen	App\Http\Controllers\Dis...	awaiting	29-06-2023 15:01	
Camiel Luijpen	App\Http\Controllers\Dis...	awaiting	29-06-2023 08:16	
Camiel Luijpen	App\Http\Controllers\Co...	succeeded	29-06-2023 06:57	
Camiel Luijpen	App\Http\Controllers\Co...	succeeded	29-06-2023 06:55	
Camiel Luijpen	App\Http\Controllers\Co...	succeeded	28-06-2023 14:06	
Camiel Luijpen	App\Http\Controllers\Co...	succeeded	28-06-2023 14:05	
Camiel Luijpen	App\Http\Controllers\Co...	succeeded	28-06-2023 14:05	
Camiel Luijpen	App\Http\Controllers\Co...	succeeded	28-06-2023 14:04	
Camiel Luijpen	App\Http\Controllers\Co...	succeeded	28-06-2023 14:04	
Camiel Luijpen	App\Http\Controllers\Ins...	succeeded	28-06-2023 10:02	

Figure 4.25: Process list interface

Figure 4.25 illustrates a detailed process list, showcasing the various processes and their corresponding status, enabling effective monitoring and management of the system's ongoing operations.

4.4.5.8 AWX jobs running in the background

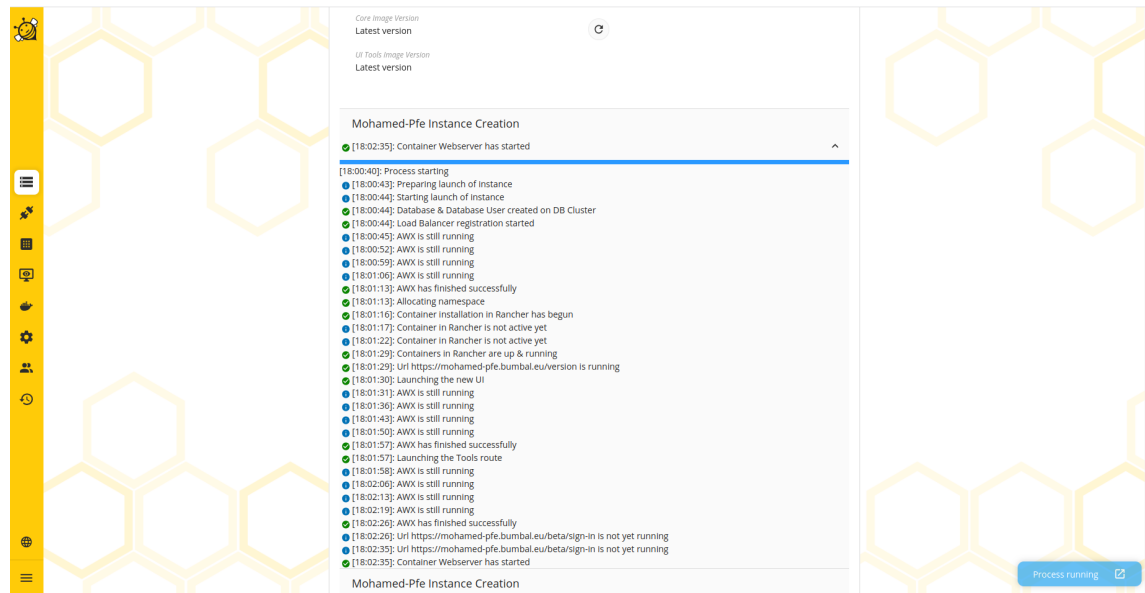


Figure 4.26: remaining jobs running in backoffice

Figure 4.26 illustrates the execution of AWX jobs in the background, showcasing the efficient processing of tasks.

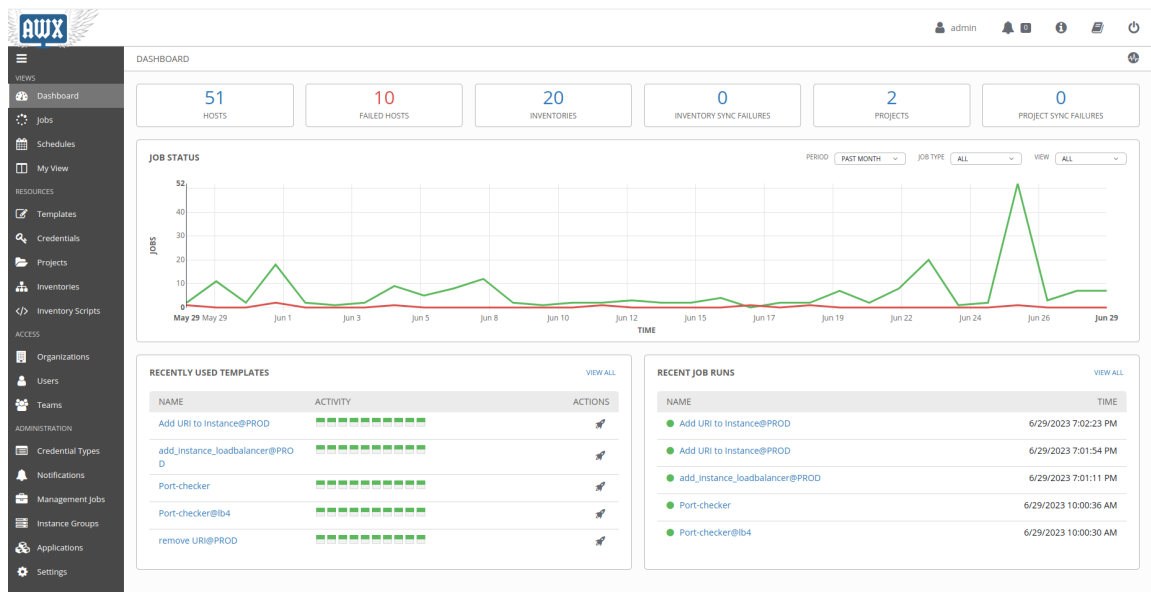


Figure 4.27: AWX dashboard interface

Figure 4.27 shows the AWX interface, providing users with an intuitive platform to manage and execute automation tasks. Through the AWX interface, users can easily schedule, monitor, and analyze various job executions.

Conclusion

In this last chapter, we defined software and hardware environments and development tools. We illustrated our system physical architecture using UML deployment diagram. And to describe the acheived work, we exposed some of the user interfaces of our solution.

General conclusion

This document is a presentation of the work carried out during our end-of-studies internship within the company Bumbal. The project aims the transition from traditional server deployment to a containerized environment by implementing a DevOps chain containing continuous integration, continuous deployment, continuous testing, continuous monitoring, continuous testing, and database clusters. Through a robust CI pipelines, we streamline code integration and automate quality tests to ensure adherence to standards. By integrating the Snyk service, we prioritize vulnerability scanning and mitigation for secure deployments. The introduction of a Continuous Deployment stage accelerates delivery, while a comprehensive monitoring system enables proactive issue detection and resource optimization. Additionally, database clusters provide enhanced performance, fault tolerance, and scalability through distributed systems and data replication. Understanding these concepts empowers us to implement efficient, secure, and scalable solutions.

This end-of-study project proved to be highly valuable to me from both theoretical and practical perspectives. It served as a gateway to the professional world, allowing me to better understand and identify the challenges of designing and deploying an application. Additionally, it provided me with hands-on experience in implementing industry-standard practices such as CI/CD, continuous monitoring, and continuous testing. Through close collaboration with the Bumbal team, which comprised members from diverse countries, I not only leveraged their experience and expertise in tackling the most challenging situations but also gained exposure to an international culture. This internship provided me with the opportunity to enhance my communication skills and adapt to a new working environment that embraced the richness and diversity of different cultures, transcending the academic framework.

CONCLUSION AND PERSPECTIVES

As perspectives, our project could be enhanced through the integration of cloud services and AI mechanisms in the DevOps tasks. This would allow us to leverage the power of cloud computing and artificial intelligence to optimize various aspects of our application, such as intelligent scaling, predictive analytics for performance monitoring, and advanced anomaly detection for proactive issue resolution.

Netography

- [1] Bumbal logo. <https://www.bumbal.eu/wp-content/uploads/2021/04/bumbal-clever-by-nature-website-logokopie.png>. (Accessed on 12/03/2023).
- [2] Wat is devops? - delta-n. <https://www.delta-n.nl/devops/wat-is-devops/>. (Accessed on 03/04/2023).
- [3] Kanban boards is beneficial. here's how. <https://kissflow.com/project/agile/wip-limits-in-kanban/>. (Accessed on 12/04/2023).
- [4] Traditional deployment vs virtualization vs container. <https://bikramat.medium.com/traditional-deployment-vs-virtualization-vs-container-f9b82ce98a50>. (Accessed on 21/03/2023).
- [5] Continuous integration and continuous deployment. <https://about.gitlab.com/topics/ci-cd/>. (Accessed on 18/04/2023).
- [6] Azure pipelines | microsoft learn. <https://learn.microsoft.com/en-us/>

azure/devops/pipelines/get-started/what-is-azure-pipelines?view=azure-devops. (Accessed on 18/04/2023).

- [7] Bamboo documentation | bamboo data center and server 8.2 | atlassian documentation. <https://confluence.atlassian.com/bamboo0802/bamboo-documentation-1103432583.html>. (Accessed on 18/04/2023).
- [8] Phpstorm workshop materials | phpstorm documentation. <https://www.jetbrains.com/help/phpstorm/workshop-materials.html>. (Accessed on 06/30/2023).
- [9] IntelliJ idea overview | intellij idea documentation. <https://www.jetbrains.com/help/idea/discover-intellij-idea.html>. (Accessed on 06/30/2023).
- [10] Quick start with datagrip | datagrip documentation. <https://www.jetbrains.com/help/datagrip/quick-start-with-datagrip.html>. (Accessed on 06/30/2023).
- [11] Documentation of the gnu project - gnu project - free software foundation. <https://www.gnu.org/doc/doc.html>. (Accessed on 06/30/2023).
- [12] Yaml ain't markup language (yaml™) version 1.1. <https://yaml.org/spec/1.1/>. (Accessed on 06/30/2023).
- [13] The apache groovy programming language - documentation. <https://groovy-lang.org/documentation.html>. (Accessed on 06/30/2023).
- [14] What is infrastructure as code. <https://www.redhat.com/en/topics/automation/what-is-infrastructure-as-code>. (Accessed on 19/04/2023).

- [15] Documentation for red hat ansible automation platform 2.3. https://access.redhat.com/documentation/en-us/red_hat_ansible_automation_platform/2.3extIdCarryOver=true&sc_cid=701f20000010H7YAAW. (Accessed on 19/04/2023).
- [16] What is ansible awx. <https://www.ansiblepilot.com/articles/what-is-ansible-awx-ansible-awx/>. (Accessed on 19/04/2023).
- [17] Docker - containerization platform. <https://aws.amazon.com/docker/>. (Accessed on 23/05/2023).
- [18] Kubernetes - container orchestration. <https://kubernetes.io/>. (Accessed on 23/05/2023).
- [19] Rancher | rancher. <https://ranchermanager.docs.rancher.com/>, month = , year = , note = (Accessed on 16/05/2023).
- [20] Beanstalk guides. <http://guides.beanstalkapp.com/version-control/intro-to-version-control.html>. (Accessed on 24/04/2023).
- [21] Jenkins user documentation. <https://www.jenkins.io/doc/>. (Accessed on 24/04/2023).
- [22] User docs - snyk user docs. <https://docs.snyk.io/>. (Accessed on 24/04/2023).
- [23] Phpstan. <https://phpstan.org/user-guide/getting-started>. (Accessed on 24/04/2023).
- [24] Mariadb knowledge base. <https://mariadb.com/kb/en/>. (Accessed on 18/03/2023).

- [25] Overview of galera cluster. <https://galeracluster.com/library/documentation/overview.html>. (Accessed on 24/04/2023).
- [26]
- [27] The tig stack in iiot/ot | influxdata. <https://www.influxdata.com/blog/tig-stack-iiot-ot/>. (Accessed on 06/12/2023).
- [28] What is a webhook? <https://www.redhat.com/en/topics/automation/what-is-a-webhook>. (Accessed on 18/04/2023).
- [29] Haproxy version - management guide. <https://docs.haproxy.org/2.8/management.html>. (Accessed on 24/04/2023).
- [30] About discord | our mission and values. <https://discord.com/company>. (Accessed on 06/12/2023).

Annexe

Traditional Server Deployment

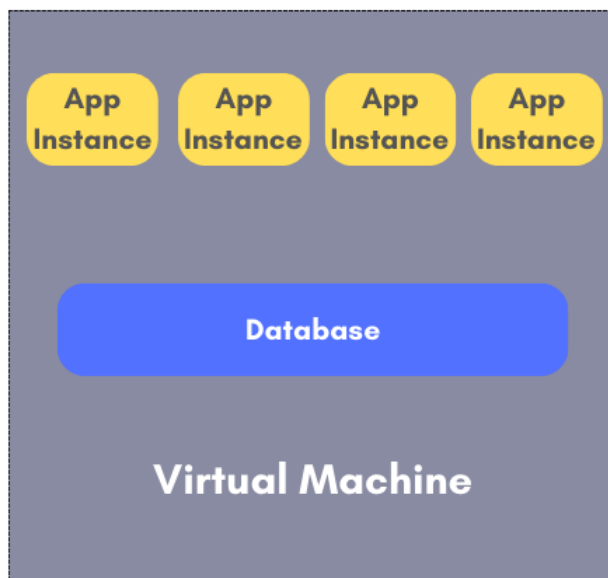


Figure A1: Traditional Server Deployment [4]

The traditional approach of the server deployment involves installing an operating

system and software applications directly onto physical hardware. This method requires significant manual intervention and is prone to server configuration drift and inconsistencies.

Drawbacks of Traditional Deployment :

- Problem with resource isolation: applications on a physical server cannot have clear boundaries when using resources.
- Limited control over resource allocation can cause conflicts between applications.
- Difficulties in scaling certain applications and causing extended periods of downtime.
- Overusing resources by one particular app can lead to the entire server crash.
- Maintaining numerous physical servers is costly for organizations.

Containerized Environment

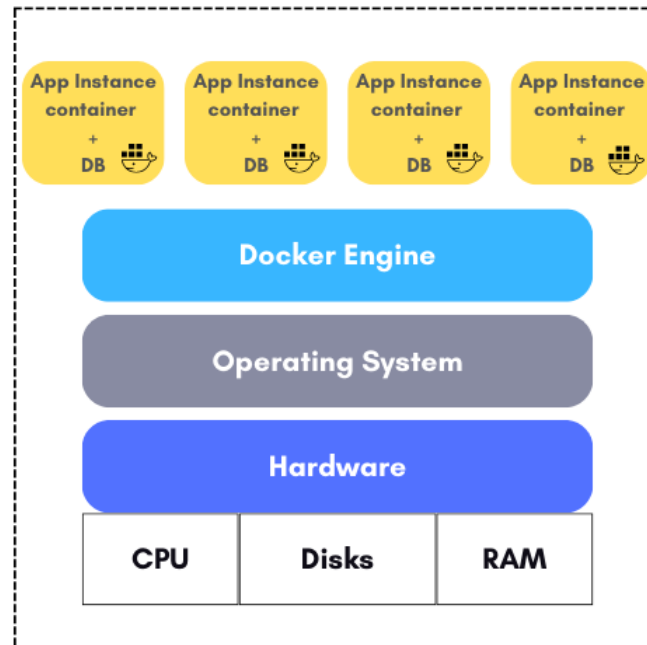


Figure A2: Docker Containers [4]

In contrast to traditional server deployment, a containerized environment offers a more efficient and streamlined approach. Applications are encapsulated within containers, which include all the necessary dependencies and libraries. This eliminates the need for manual OS installation and ensures consistent configurations across different environments.

Advantages of Containerization :

- If an application instance is running in a container, any failure or downtime experienced by that instance will not impact other instances.
- Containers encapsulate both the code and its required dependencies, providing a portable and isolated environment for executing applications on various

operating systems.

- Containers are smaller in size compared to virtual machines and start up quickly.
- Containers leverage hardware resources effectively, ensuring optimal utilization without any resource wastage.
- Containers are very lightweight and fast. They can be ready to use in just a few seconds or even milliseconds.
- Docker containers are isolated from other processes and don't require special hardware.
- Containers can be easily moved from one operating system to another.
- Containers can be scaled up or down easily using orchestration platforms like Kubernetes or Docker Swarm, which helps creating and managing Docker containers.

Comparison between the deployment types The aim is to compare several features such as scalability, availability, maintenance, cost, and flexibility for each method, Table A1 illustrate this comparison.

Deployment Feature	Traditional Deployment	Containers
Scalability	Limited by hardware	Horizontal/Vertical scaling
Availability	Single point of failure	Highly available
Resource Utilization	Inefficient	Optimized
Deployment	Manual	Automated
Management	Manual	Automated

Table A1: Comparison between Traditional Server Deployment and Containers

